US 20250279091A1

(19) United States

(12) Patent Application Publication (10) Pub. No.: US 2025/0279091 A1

Bataev et al. (43) Pub. Date: Sep. 4, 2025

(54) **LABEL-LOOPING PREDICTION FOR AUTOMATIC SPEECH RECOGNITION AND OTHER AI SYSTEMS**

(71) Applicant: **NVIDIA Corporation**, Santa Clara, CA (US)

(72) Inventors: **Vladimir Bataev**, Yerevan (AM); **Hainan Xu**, Beijing (CN); **Vitaly Lavrukhin**, Campbell, CA (US); **Boris Ginsburg**, Sunnyvale, CA (US)

(21) Appl. No.: **18/820,028**

(22) Filed: **Aug. 29, 2024**

Related U.S. Application Data

(60) Provisional application No. 63/559,586, filed on Feb. 29, 2024.

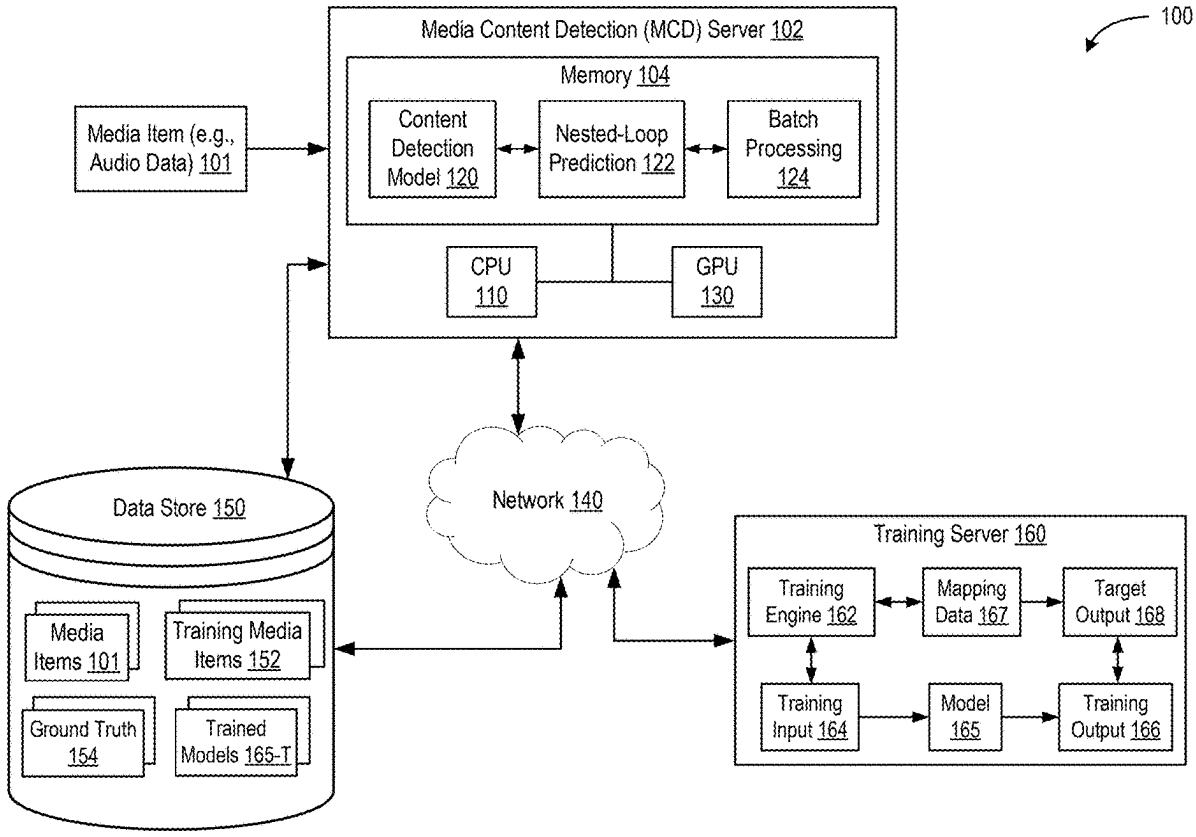
Publication Classification

(51) **Int. Cl.**
G10L 15/16 (2006.01)

(52) **U.S. Cl.**
CPC ***G10L 15/16*** (2013.01)

(57) **ABSTRACT**

Disclosed are apparatuses, systems, and techniques that use label-looping processing for efficient automatic speech recognition (ASR). The techniques include performing a plurality of iterations of an outer processing loop to identify content units (CUs) of a media item having multiple frames. An individual iteration of the outer processing loop includes updating, using a first neural network (NN) and identified non-blank CU, a state of the media item and performing one or more iterations of an inner processing loop. An individual iteration of the inner processing loop includes processing, using a second NN, the state of the media item and an individual frame to predict a CU associated with the individual frame. The iterations of the inner processing loop are performed until the predicted CU corresponds to a non-blank CU. The identified plurality of CUs is used to generate a representation of the media item.



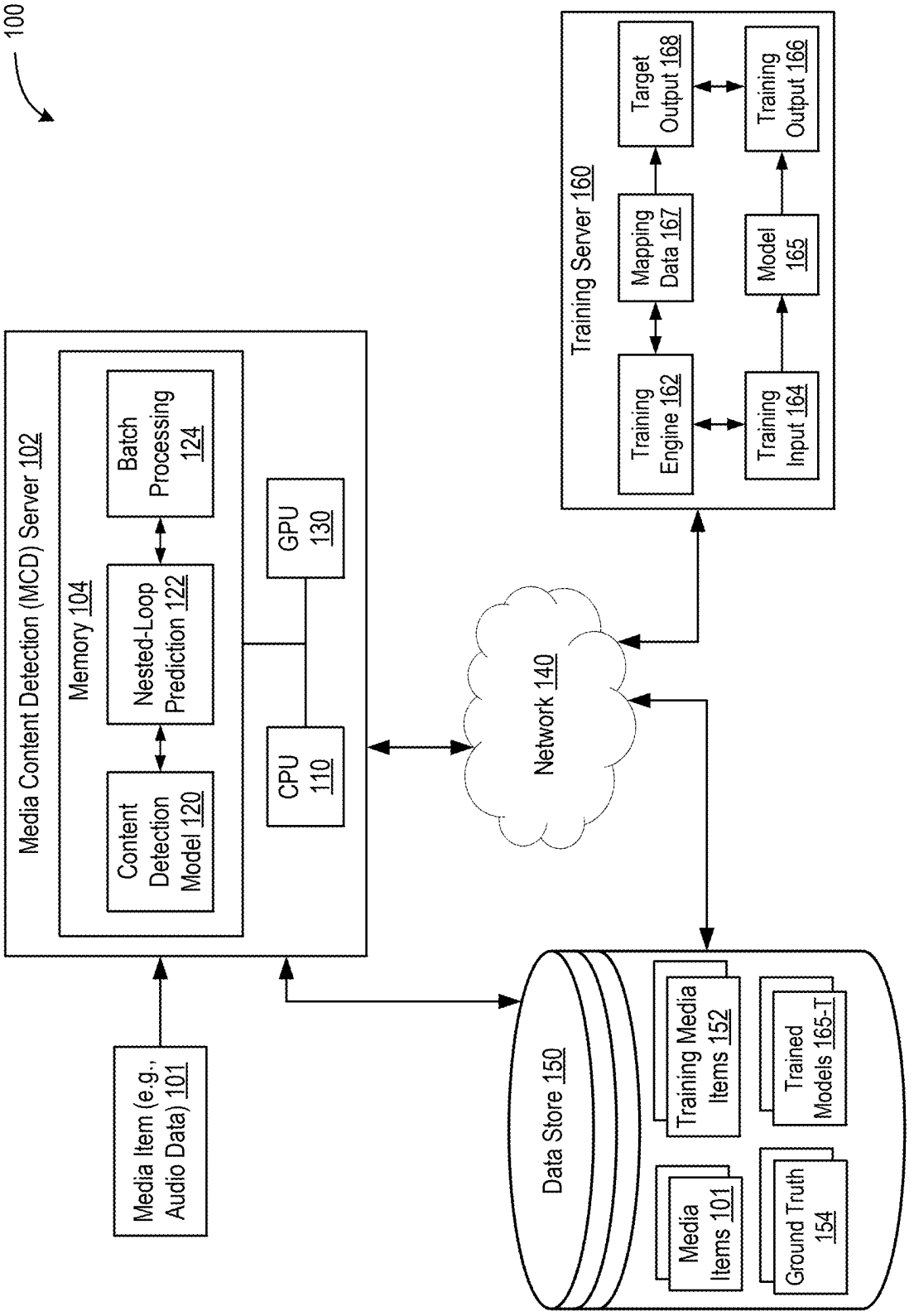

100
Media Content Detection (MCD) Server 102
Memory 104
Content Detection Model 120
Nested-Loop Prediction 122
Batch Processing 124
CPU 110
GPU 130
Network 140
Training Server 160
Training Engine 162
Training Input 164
Model 165
Mapping Data 167
Target Output 168
Training Output 166
Media Item (e.g., Audio Data) 101
Data Store 150
Media Items 101
Training Media Items 152
Ground Truth 154
Trained Models 165-T

FIG. 1A

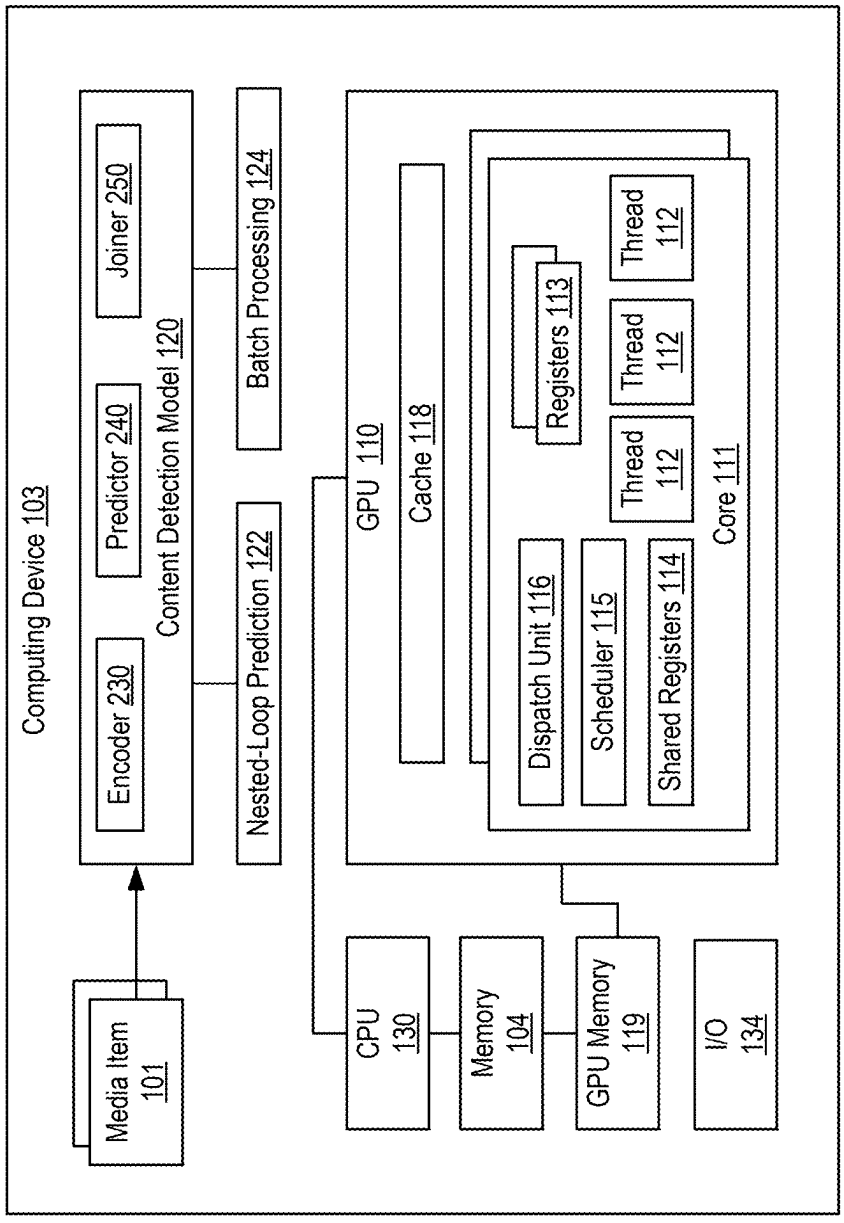

Computing Device 103
Media Item 101
Content Detection Model 120
Encoder 230
Predictor 240
Joiner 250
Nested-Loop Prediction 122
Batch Processing 124
GPU 110
Cache 118
Core 111
Dispatch Unit 116
Scheduler 115
Shared Registers 114
Registers 113
Thread 112
Thread 112
Thread 112
Thread 112
CPU 130
Memory 104
GPU Memory 119
I/O 134

FIG. 1B

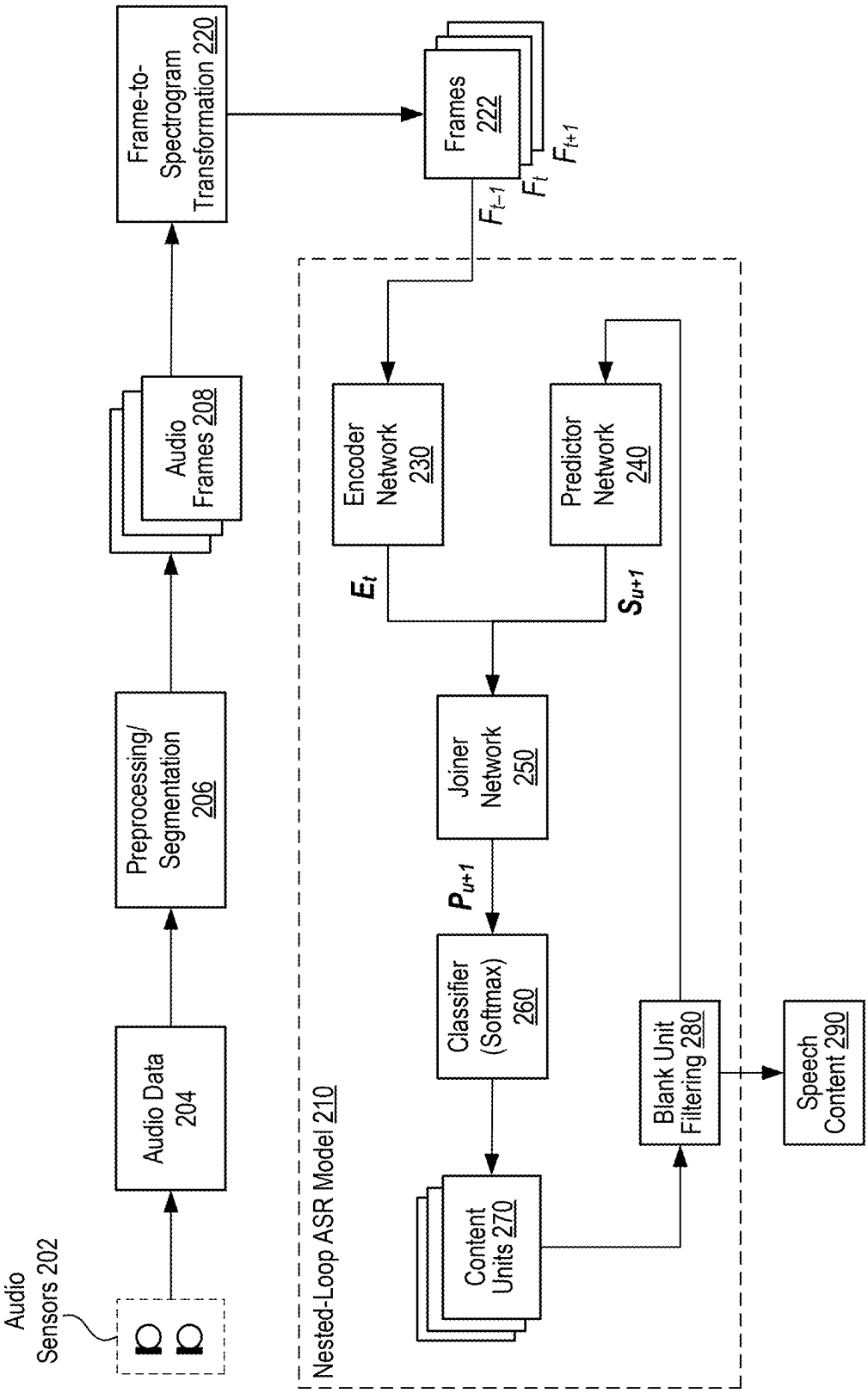

200
Audio Sensors 202
Audio Data 204
Preprocessing/ Segmentation 206
Audio Frames 208
Frame-to-Spectrogram Transformation 220
Frames 222
F_{t-1}
F_t
F_{t+1}
Nested-Loop ASR Model 210
Encoder Network 230
Predictor Network 240
E_t
S_{u+1}
Joiner Network 250
P_{u+1}
Classifier (Softmax) 260
Content Units 270
Blank Unit Filtering 280
Speech Content 290

FIG. 2

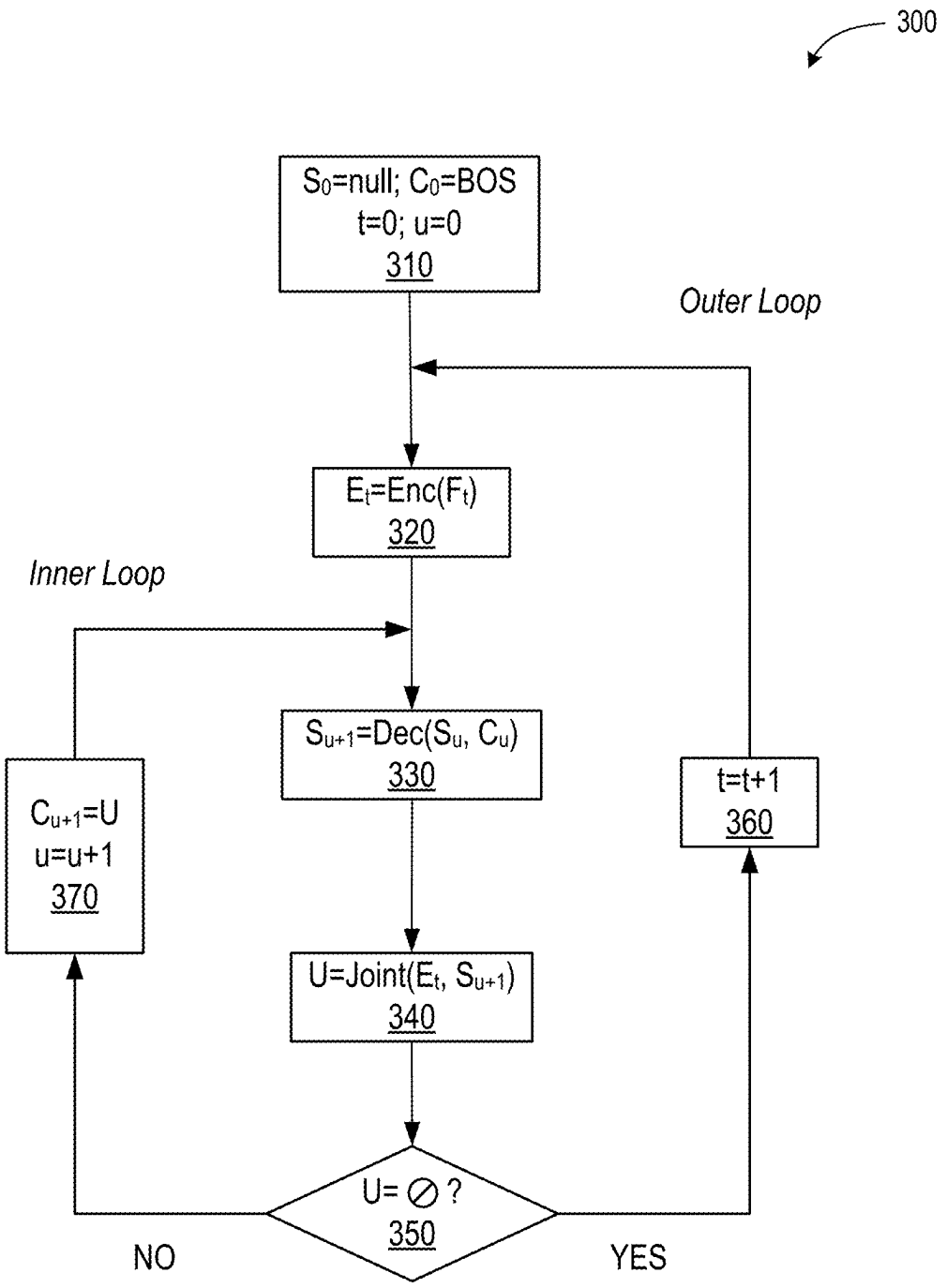

300
S_0=null; C_0=BOS
t=0; u=0
310
Outer Loop
E_t=Enc(F_t)
320
Inner Loop
S_{u+1}=Dec(S_u, C_u)
330
C_{u+1}=U
u=u+1
370
t=t+1
360
U=Joint(E_t, S_{u+1})
340
U= ⊘ ?
350
NO
YES

FIG. 3

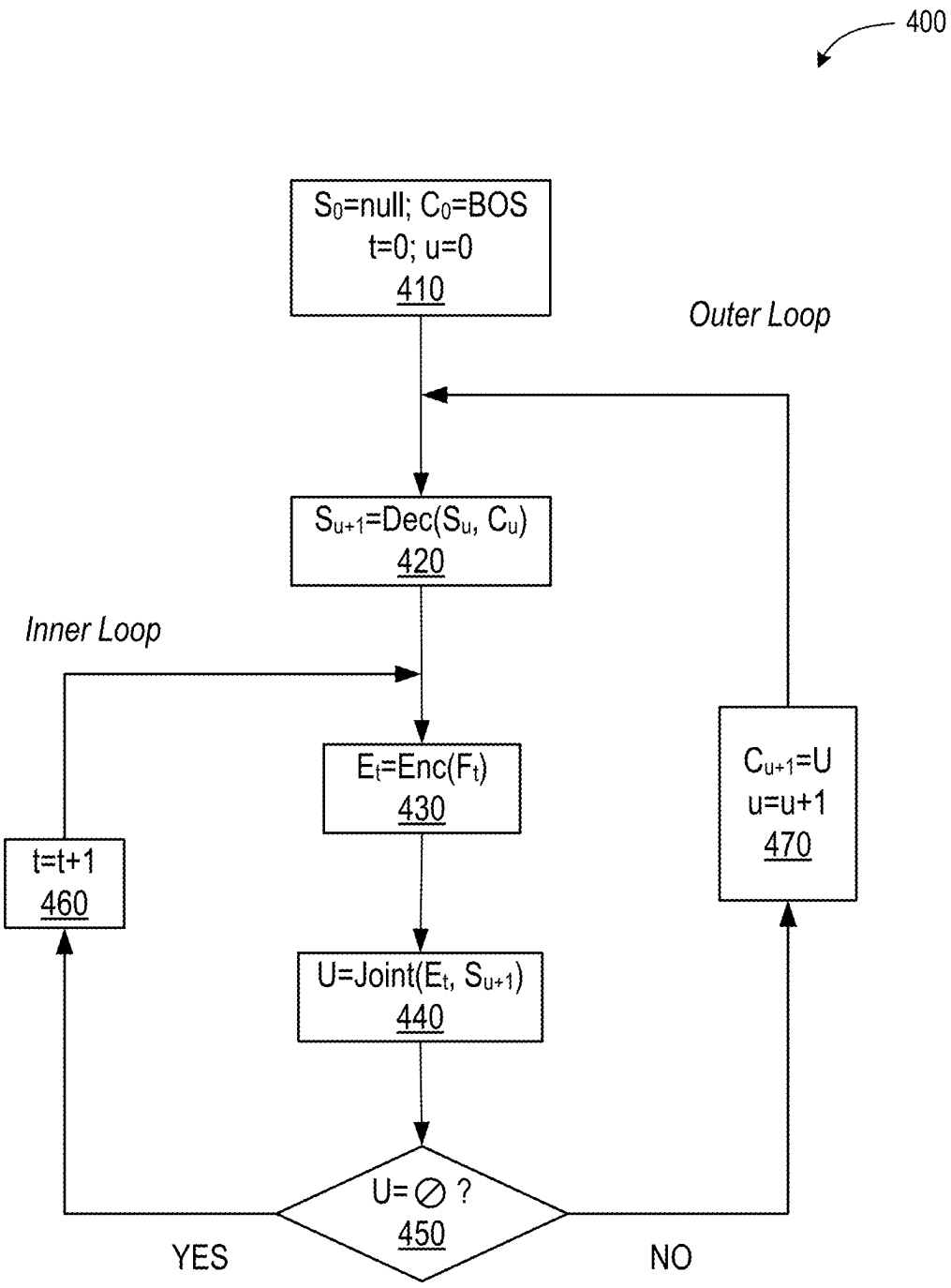

400
S_0=null; C_0=BOS
t=0; u=0
410
Outer Loop
S_{u+1}=Dec(S_u, C_u)
420
Inner Loop
E_t=Enc(F_t)
430
C_{u+1}=U
u=u+1
470
t=t+1
460
U=Joint(E_t, S_{u+1})
440
U=∅ ?
450
YES
NO

FIG. 4

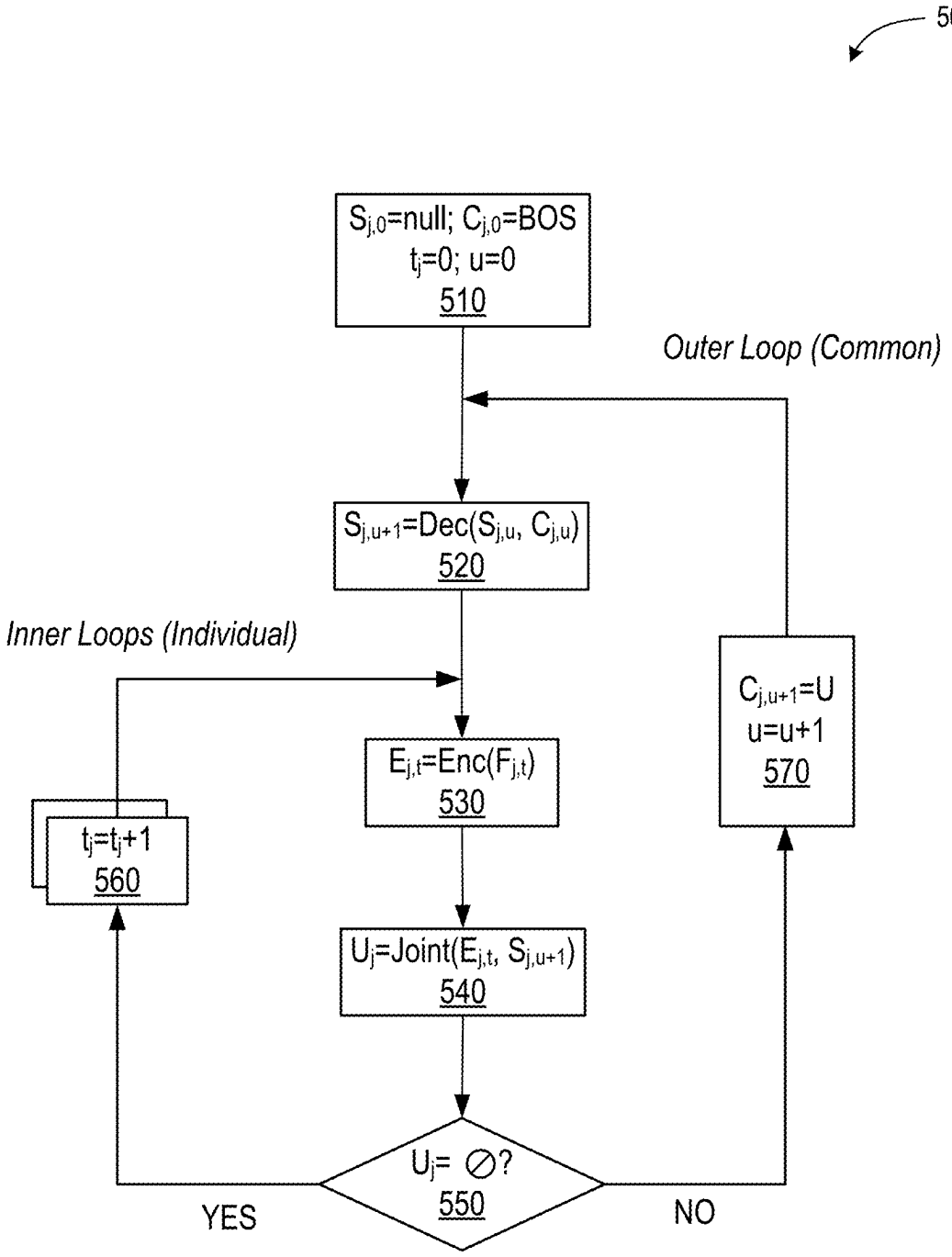

500
$S_{j,0}$=null; $C_{j,0}$=BOS
t_j=0; u=0
510
Outer Loop (Common)
$S_{j,u+1}$=Dec($S_{j,u}$, $C_{j,u}$)
520
Inner Loops (Individual)
$E_{j,t}$=Enc($F_{j,t}$)
530
t_j=t_j+1
560
U_j=Joint($E_{j,t}$, $S_{j,u+1}$)
540
U_j= ∅?
550
YES
NO
$C_{j,u+1}$=U
u=u+1
570

FIG. 5

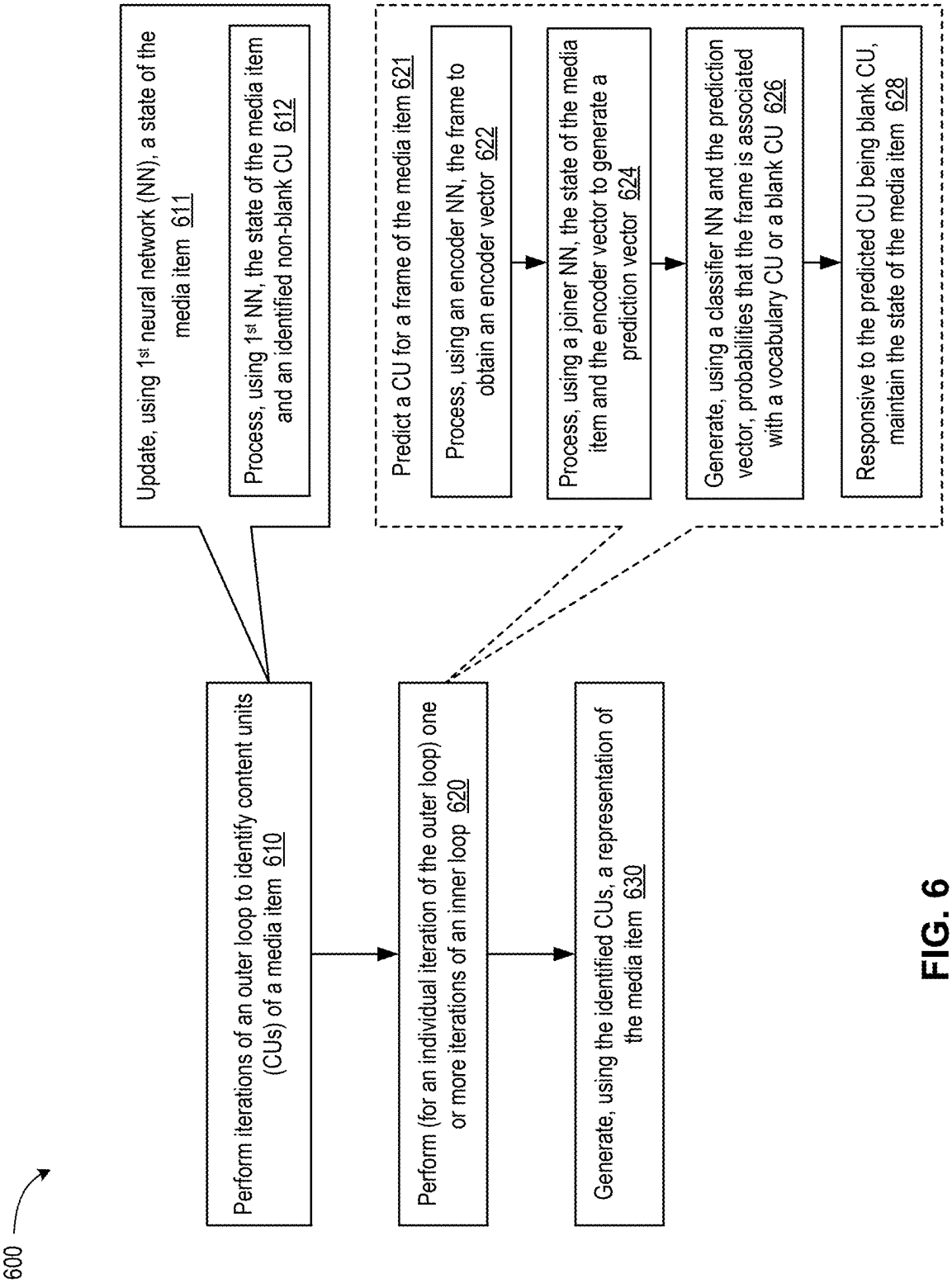

600
Perform iterations of an outer loop to identify content units (CUs) of a media item 610
Update, using 1st neural network (NN), a state of the media item 611
Process, using 1st NN, the state of the media item and an identified non-blank CU 612
Perform (for an individual iteration of the outer loop) one or more iterations of an inner loop 620
Predict a CU for a frame of the media item 621
Process, using an encoder NN, the frame to obtain an encoder vector 622
Process, using a joiner NN, the state of the media item and the encoder vector to generate a prediction vector 624
Generate, using a classifier NN and the prediction vector, probabilities that the frame is associated with a vocabulary CU or a blank CU 626
Responsive to the predicted CU being blank CU, maintain the state of the media item 628
Generate, using the identified CUs, a representation of the media item 630

FIG. 6

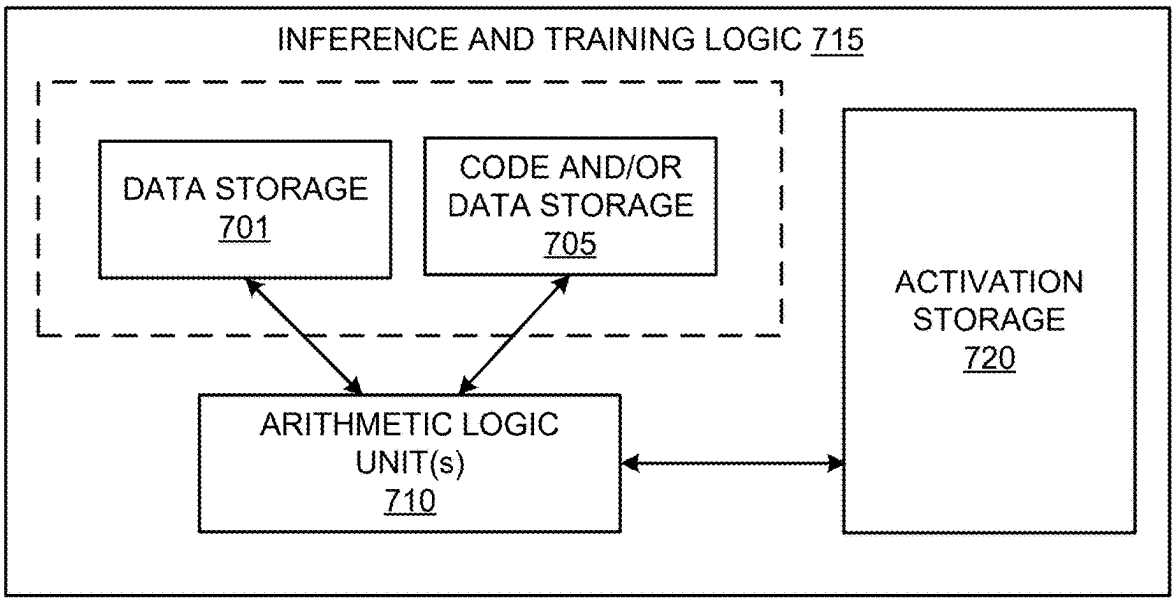

INFERENCE AND TRAINING LOGIC 715
DATA STORAGE 701
CODE AND/OR DATA STORAGE 705
ACTIVATION STORAGE 720
ARITHMETIC LOGIC UNIT(s) 710

FIG. 7A

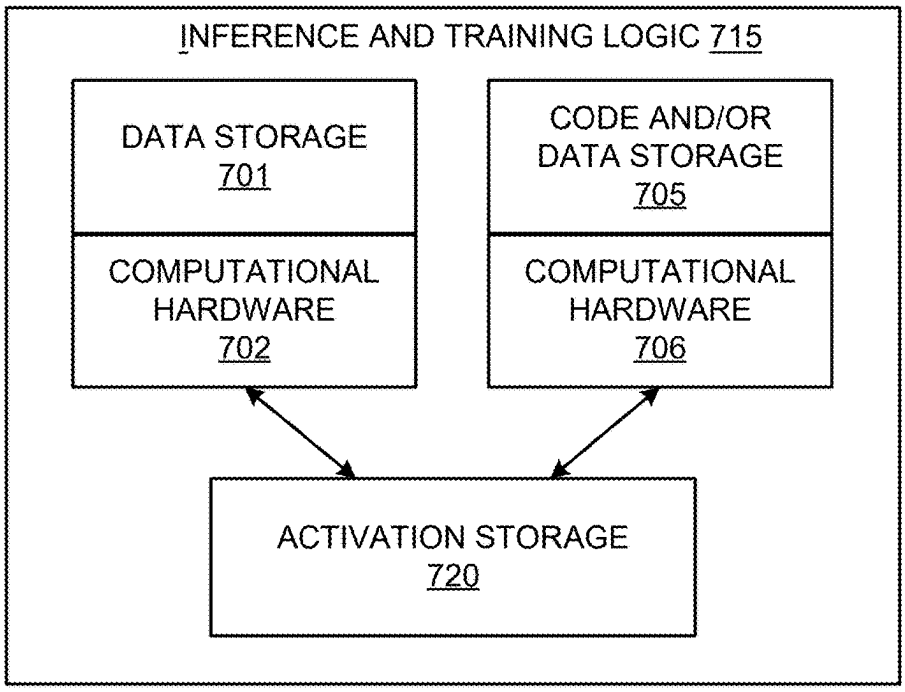

INFERENCE AND TRAINING LOGIC 715
DATA STORAGE 701
CODE AND/OR DATA STORAGE 705
COMPUTATIONAL HARDWARE 702
COMPUTATIONAL HARDWARE 706
ACTIVATION STORAGE 720

FIG. 7B

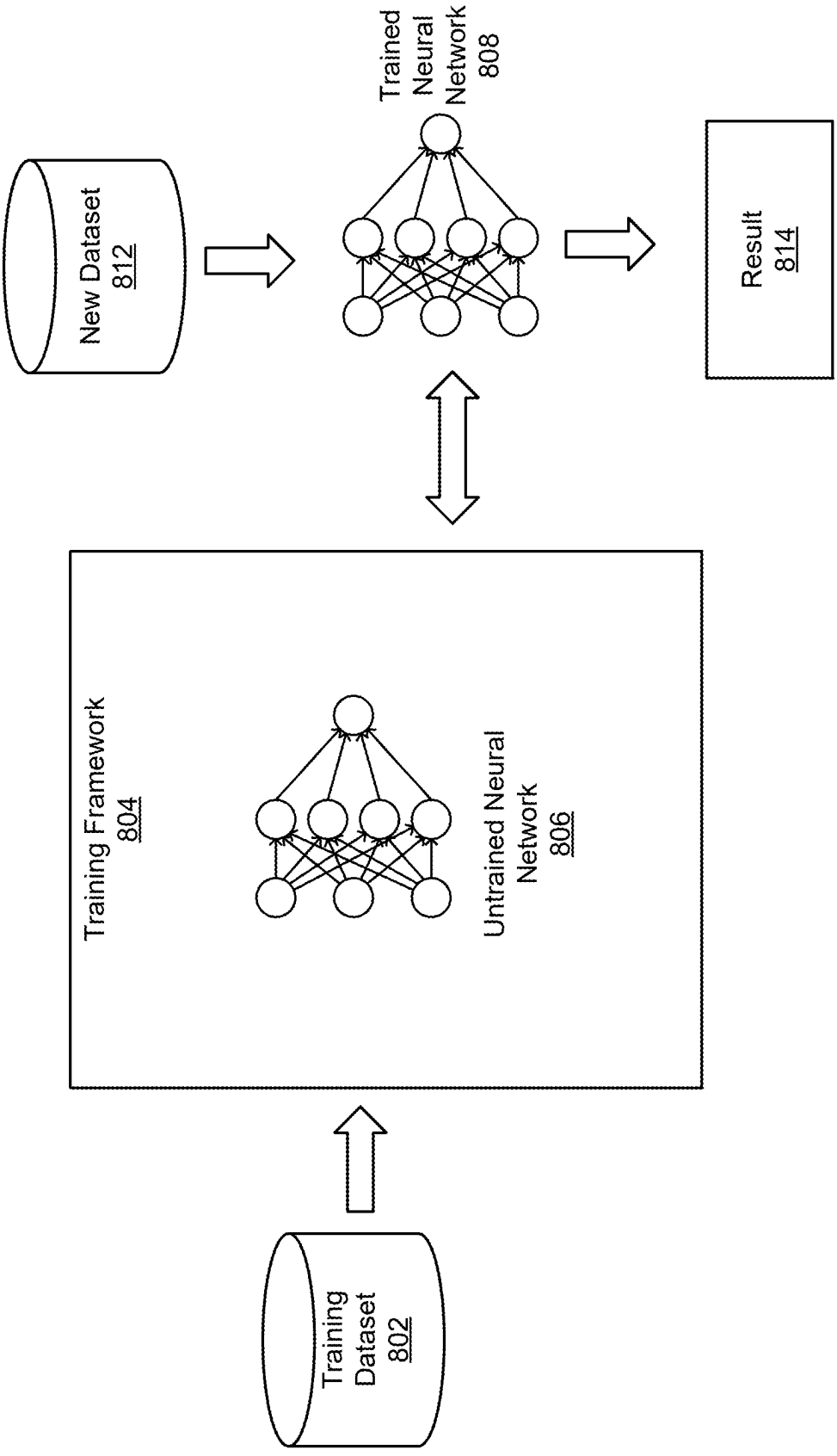

Training Dataset
802
Training Framework
804
Untrained Neural Network
806
Trained Neural Network
808
New Dataset
812
Result
814

FIG. 8

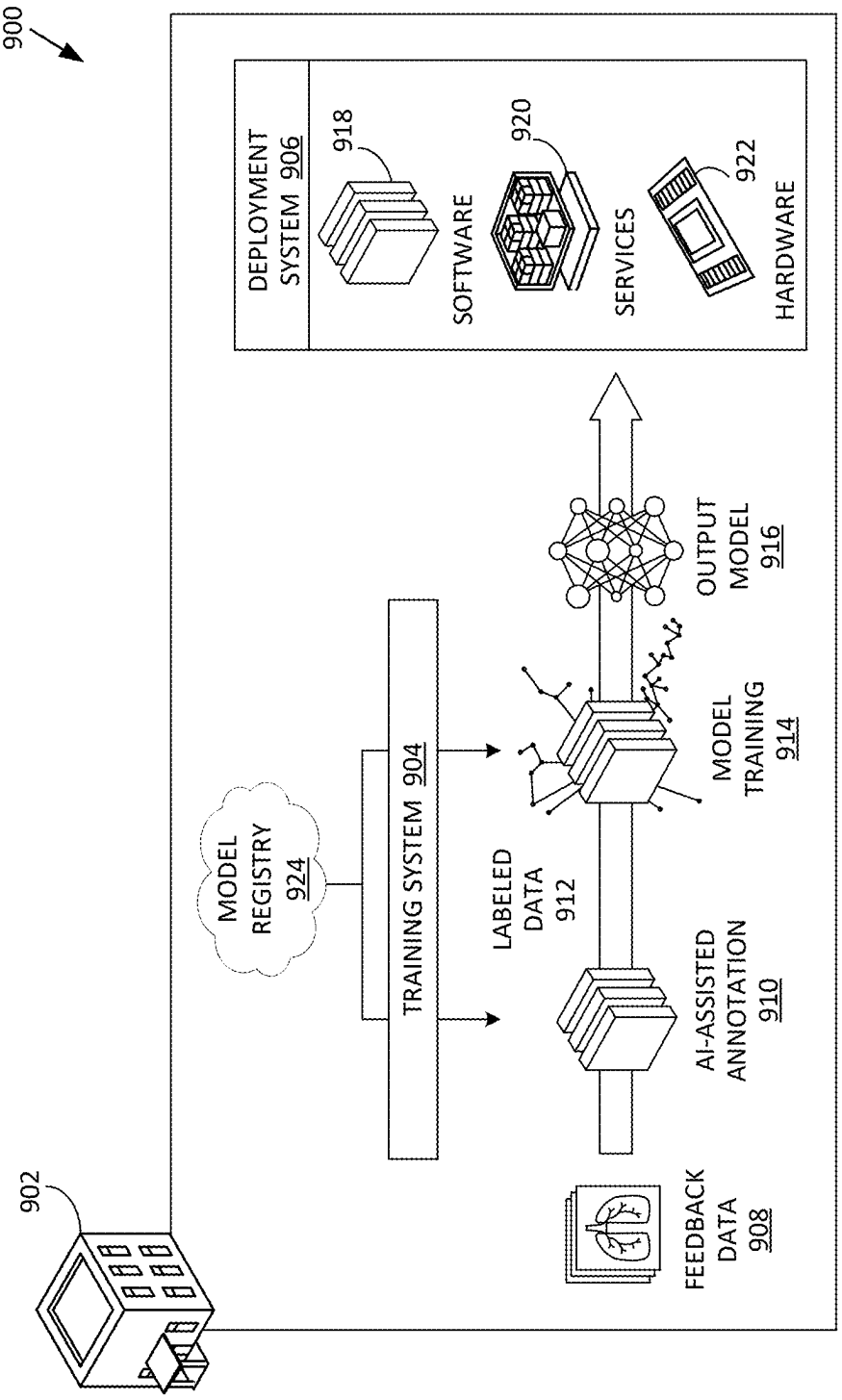

900
902
DEPLOYMENT SYSTEM 906
918
SOFTWARE
920
SERVICES
922
HARDWARE
OUTPUT MODEL 916
MODEL TRAINING 914
LABELED DATA 912
AI-ASSISTED ANNOTATION 910
FEEDBACK DATA 908
TRAINING SYSTEM 904
MODEL REGISTRY 924

FIG. 9

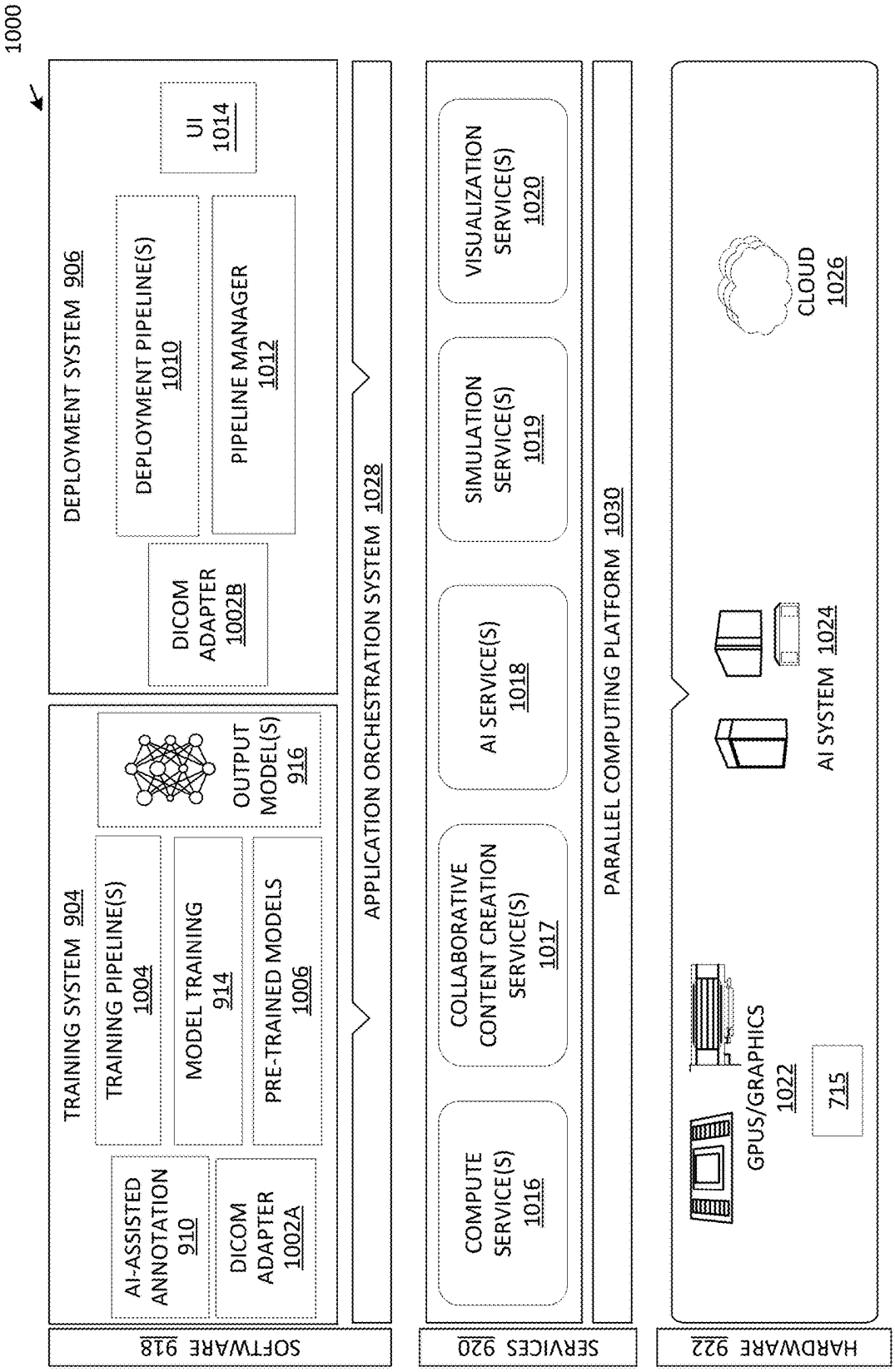

1000
SOFTWARE 918
TRAINING SYSTEM 904
AI-ASSISTED ANNOTATION 910
DICOM ADAPTER 1002A
TRAINING PIPELINE(S) 1004
MODEL TRAINING 914
PRE-TRAINED MODELS 1006
OUTPUT MODEL(S) 916
DEPLOYMENT SYSTEM 906
DICOM ADAPTER 1002B
DEPLOYMENT PIPELINE(S) 1010
PIPELINE MANAGER 1012
UI 1014
APPLICATION ORCHESTRATION SYSTEM 1028
SERVICES 920
COMPUTE SERVICE(S) 1016
COLLABORATIVE CONTENT CREATION SERVICE(S) 1017
AI SERVICE(S) 1018
SIMULATION SERVICE(S) 1019
VISUALIZATION SERVICE(S) 1020
PARALLEL COMPUTING PLATFORM 1030
HARDWARE 922
GPUS/GRAPHICS 1022
715
AI SYSTEM 1024
CLOUD 1026

FIG. 10

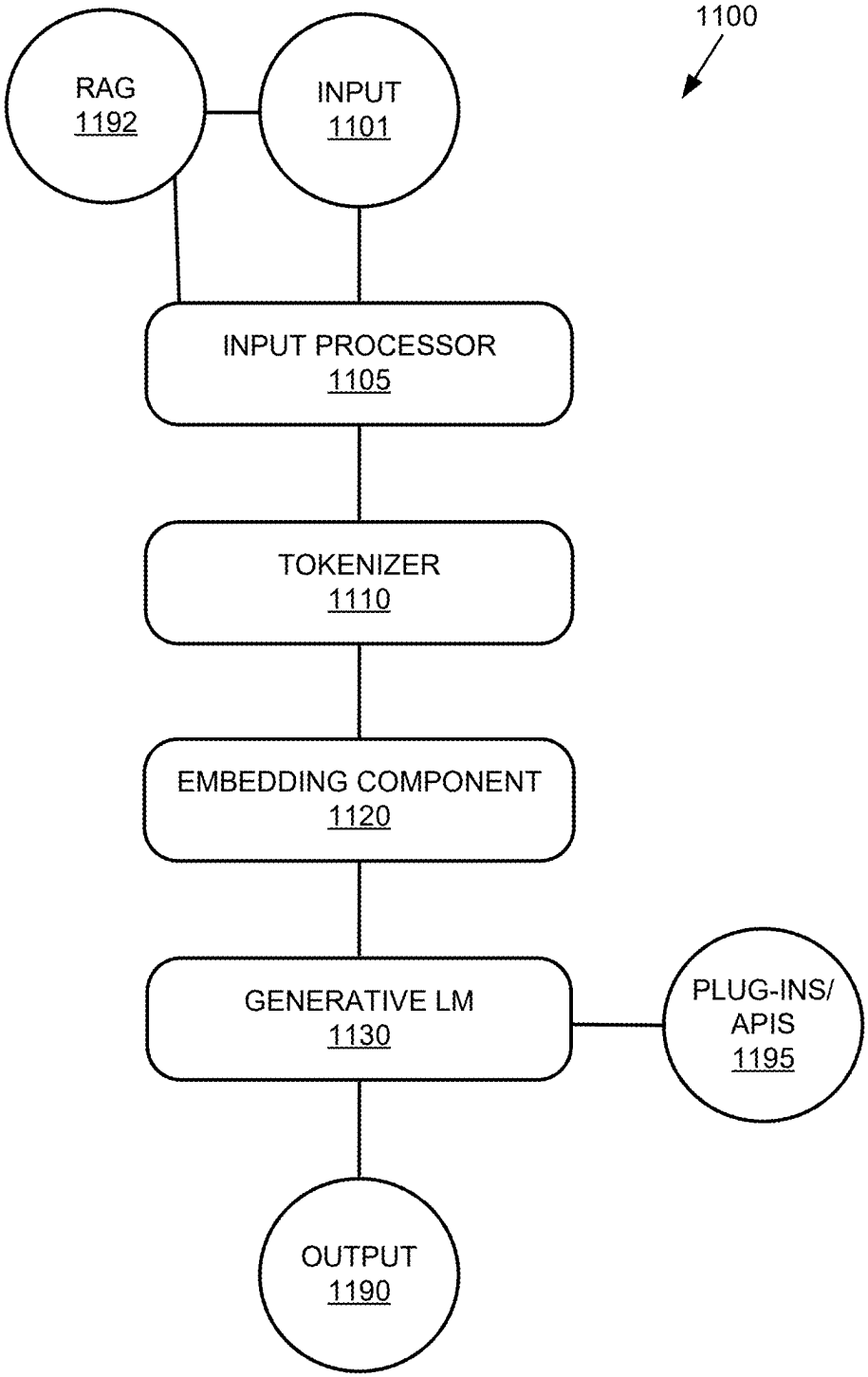

1100
RAG
1192
INPUT
1101
INPUT PROCESSOR
1105
TOKENIZER
1110
EMBEDDING COMPONENT
1120
GENERATIVE LM
1130
PLUG-INS/
APIS
1195
OUTPUT
1190

FIG. 11A

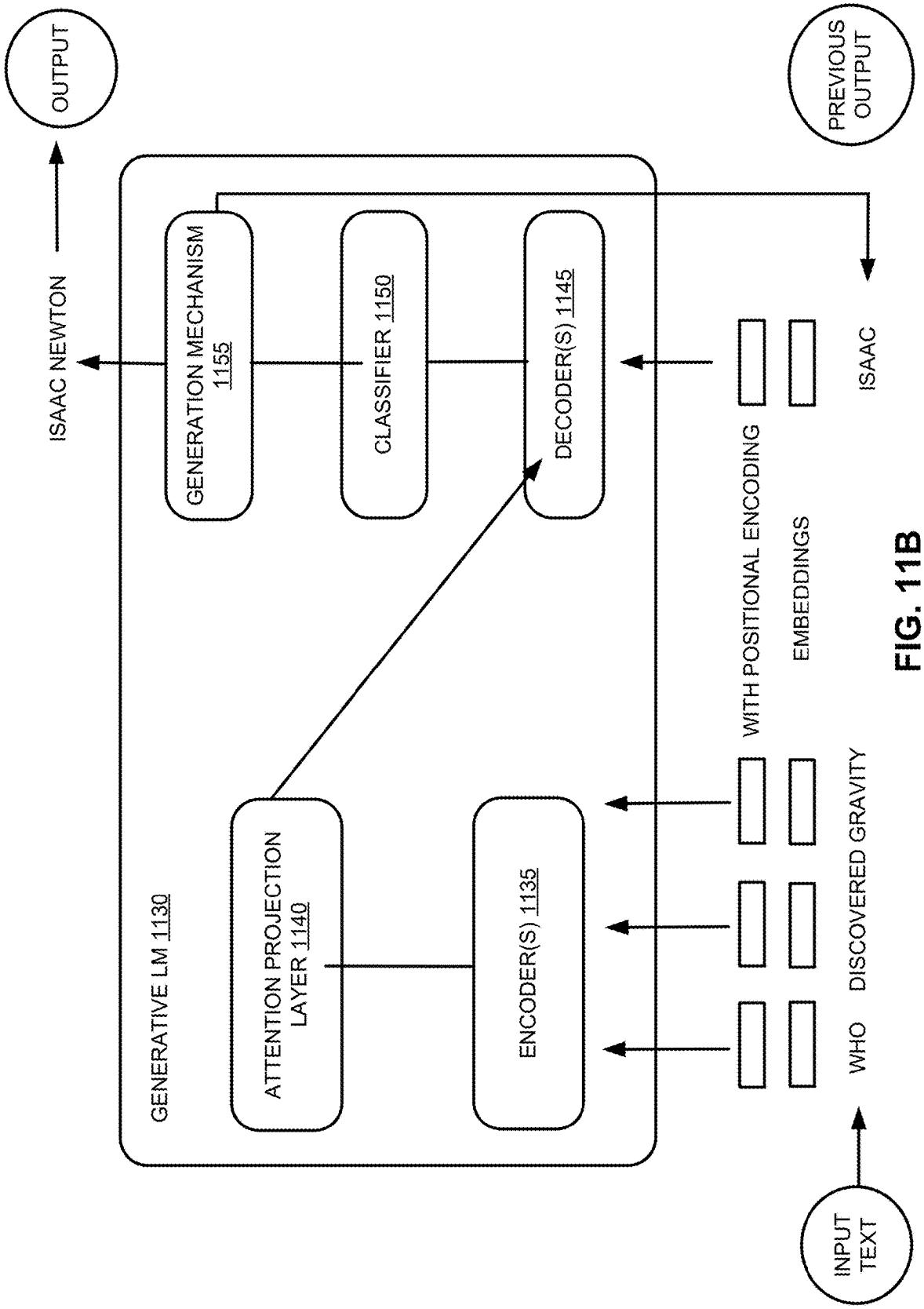

OUTPUT
PREVIOUS OUTPUT
ISAAC NEWTON
GENERATION MECHANISM 1155
CLASSIFIER 1150
DECODER(S) 1145
ISAAC
WITH POSITIONAL ENCODING
EMBEDDINGS
GENERATIVE LM 1130
ATTENTION PROJECTION LAYER 1140
ENCODER(S) 1135
WHO
DISCOVERED GRAVITY
INPUT TEXT

FIG. 11B

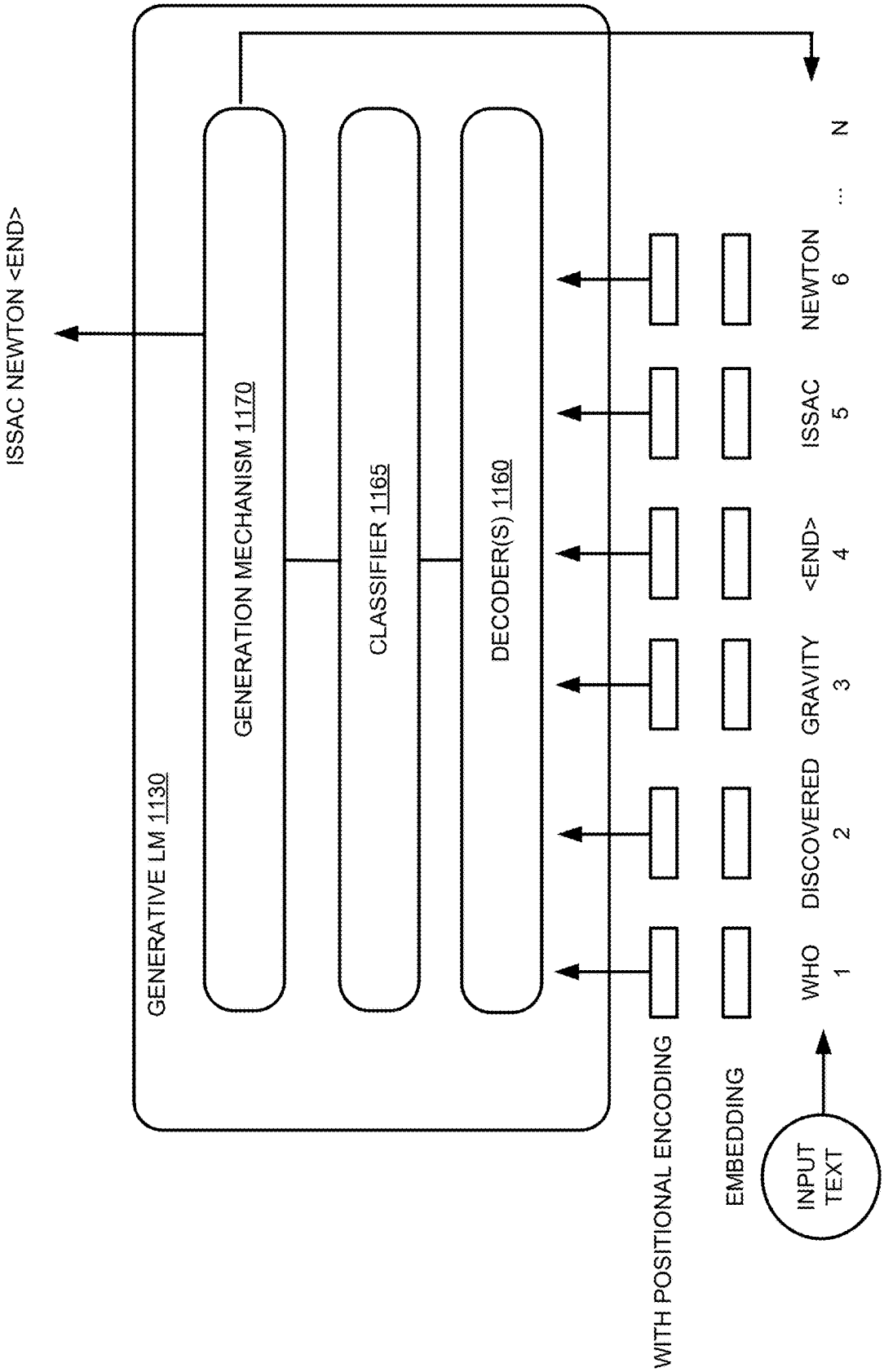

INPUT TEXT
EMBEDDING
WITH POSITIONAL ENCODING
WHO 1
DISCOVERED 2
GRAVITY 3
<END> 4
ISSAC 5
NEWTON 6
... N
GENERATIVE LM 1130
GENERATION MECHANISM 1170
CLASSIFIER 1165
DECODER(S) 1160
ISSAC NEWTON <END>

FIG. 11C

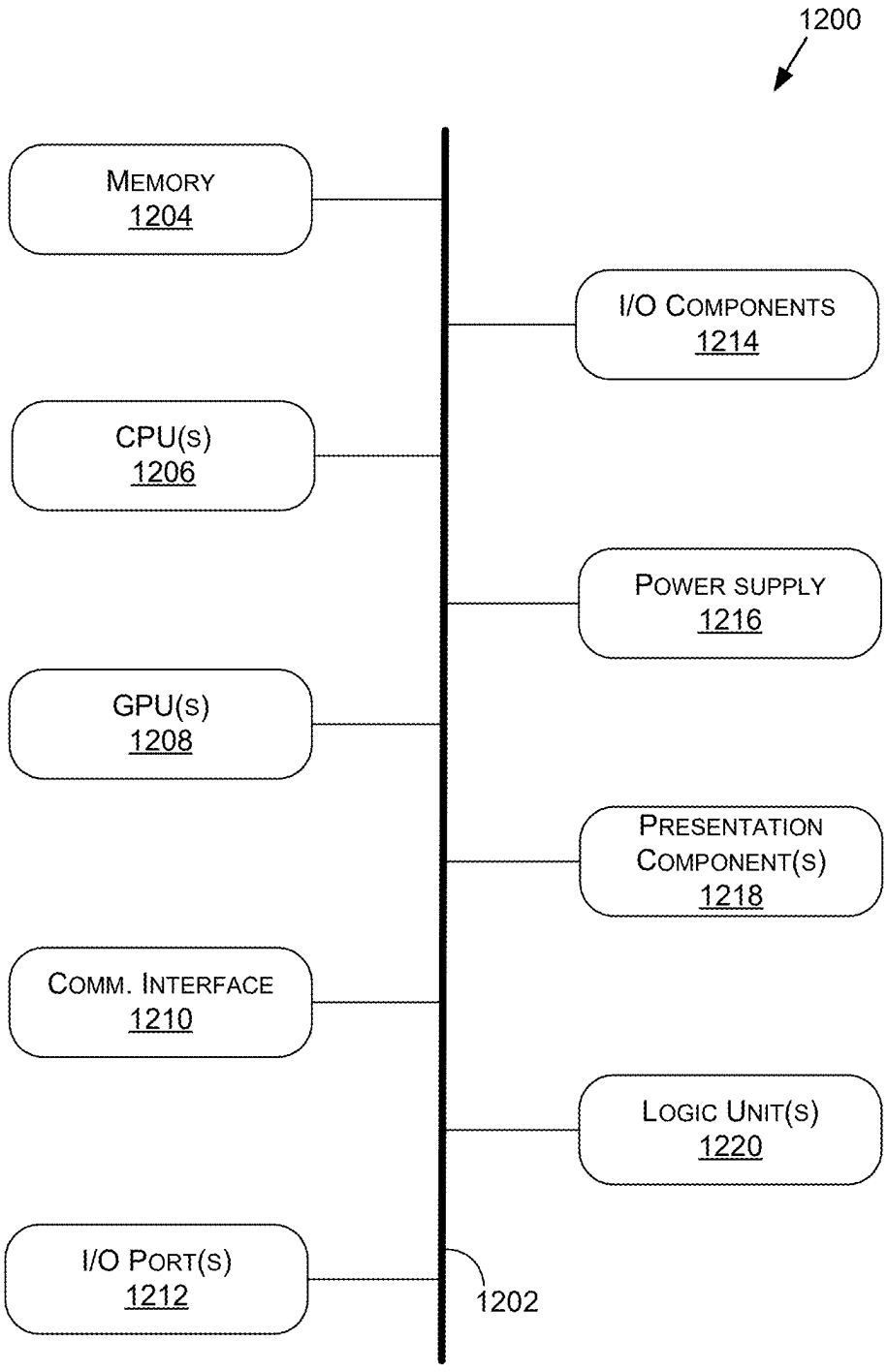

1200
MEMORY
1204
I/O COMPONENTS
1214
CPU(S)
1206
POWER SUPPLY
1216
GPU(S)
1208
PRESENTATION
COMPONENT(S)
1218
COMM. INTERFACE
1210
LOGIC UNIT(S)
1220
I/O PORT(S)
1212
1202

FIG. 12

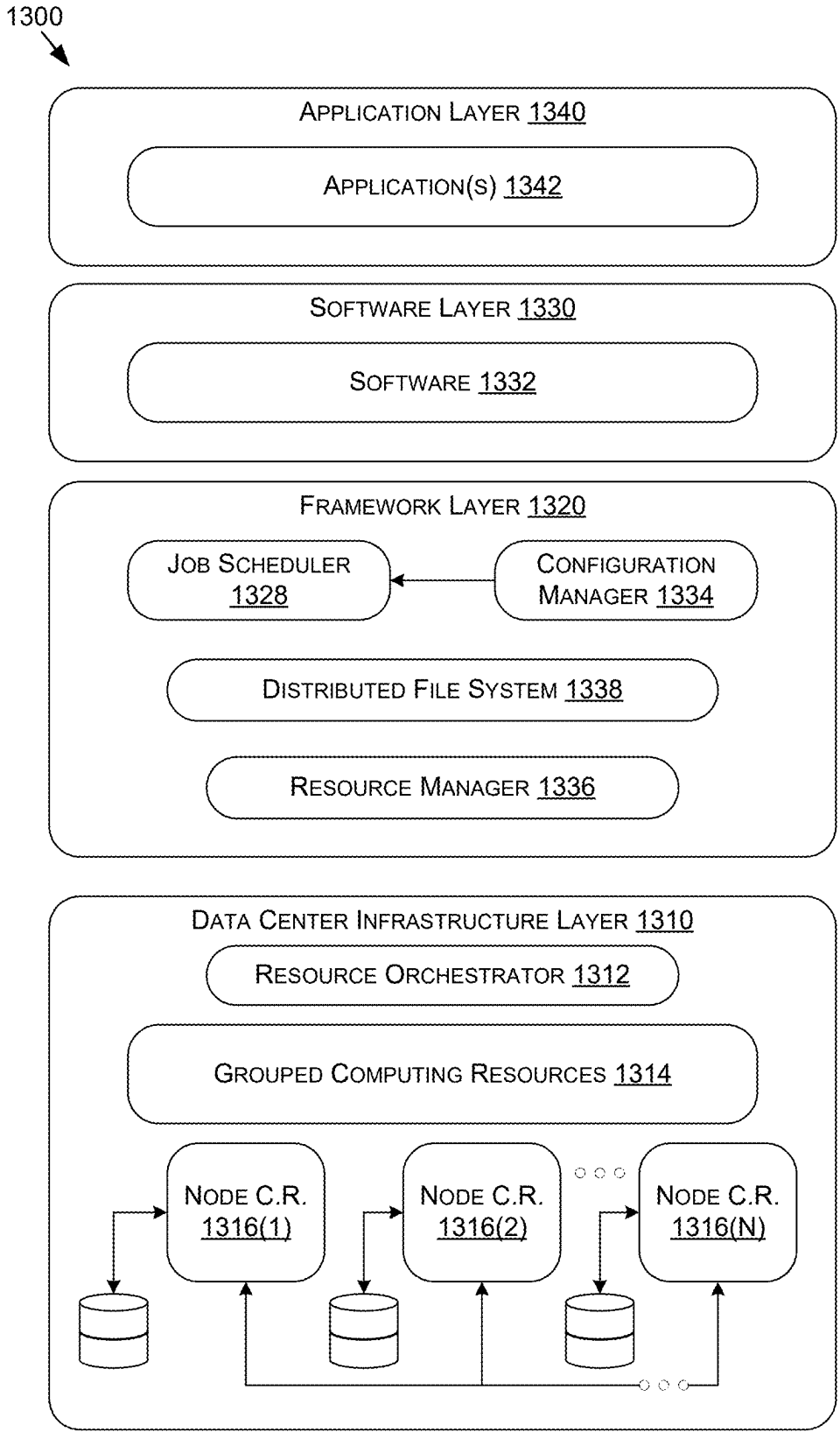

1300
APPLICATION LAYER 1340
APPLICATION(S) 1342
SOFTWARE LAYER 1330
SOFTWARE 1332
FRAMEWORK LAYER 1320
JOB SCHEDULER 1328
CONFIGURATION MANAGER 1334
DISTRIBUTED FILE SYSTEM 1338
RESOURCE MANAGER 1336
DATA CENTER INFRASTRUCTURE LAYER 1310
RESOURCE ORCHESTRATOR 1312
GROUPED COMPUTING RESOURCES 1314
NODE C.R. 1316(1)
NODE C.R. 1316(2)
NODE C.R. 1316(N)

FIG. 13

LABEL-LOOPING PREDICTION FOR AUTOMATIC SPEECH RECOGNITION AND OTHER AI SYSTEMS

RELATED APPLICATIONS

[0001] This application claims the benefit of U.S. Provisional Patent Application No. 63/559,586, filed Feb. 29, 2024, entitled "Label-Looping: Highly Efficient Decoding for Transducers," the contents of which are incorporated by reference in their entirety herein.

TECHNICAL FIELD

[0002] At least one embodiment pertains to processing resources used to perform and facilitate speech recognition, transcription, diarization, and/or text-to-speech conversion. For example, at least one embodiment pertains to systems and techniques that facilitate efficient automated speech recognition assisted with target word spotting.

BACKGROUND

[0003] Speech recognition, also known as automatic speech recognition (ASR) or speech-to-text (STT or S2T), is an intersection of computer technology and linguistics directed to techniques of recognition and translation of spoken language into text. ASR systems often deploy machine-learning models (MLMs), e.g., trained neural networks, to recognize phonemes, graphemes, words, subwords, sentences, and other units of speech. Speaker-independent ASR models rely on general phonetic and semantic characteristics of speech that remain uniform across different speakers. Speaker-dependent ASR models use samples of speech of a particular speaker to fine-tune the models to recognize that person's speech, resulting in increased accuracy of ASR processing.

[0004] Other automatic speech tasks facilitated by machine learning include speaker identification that involves associating spoken utterances with speakers whose speech samples are stored a database of speakers (or identifying a new speaker not represented in the database), speaker verification that involves determining whether two or more utterances are spoken by the same speaker or different speakers, speaker diarization that involves partitioning unstructured speech among various participants of a conversation or meeting, and other tasks.

BRIEF DESCRIPTION OF DRAWINGS

[0005] FIG. **1**A is a block diagram of an example computer system capable of performing content detection in media items using label-looping prediction, according to at least one embodiment;

[0006] FIG. **1**B illustrates an example computing device that supports training or deployment of content detection models that identify content in media items using label-looping prediction, according to at least one embodiment;

[0007] FIG. **2** illustrates an example architecture and data flow of an audio processing pipeline that implements label-looping prediction for efficient decoding of media content, according to at least one embodiment;

[0008] FIG. **3** illustrates a data flow in a conventional transducer-type ASR system that deploys frame-looping, according to at least one embodiment;

[0009] FIG. **4** illustrates an example data flow in a transducer-type ASR system that deploys label-looping prediction for efficient decoding of media content, according to at least one embodiment;

[0010] FIG. **5** illustrates an example data flow in a transducer-type ASR system that deploys label-looping prediction for batch processing of media content, according to at least one embodiment;

[0011] FIG. **6** is a flow diagram of a method of automatic content recognition that deploys label-looping prediction, according to at least one embodiment;

[0012] FIG. **7**A illustrates inference and/or training logic, according to at least one embodiment;

[0013] FIG. **7**B illustrates inference and/or training logic, according to at least one embodiment;

[0014] FIG. **8** illustrates training and deployment of a neural network, according to at least one embodiment;

[0015] FIG. **9** is an example data flow diagram for an advanced computing pipeline, according to at least one embodiment;

[0016] FIG. **10** is a system diagram for an example system for training, adapting, instantiating and deploying machine learning models in an advanced computing pipeline, according to at least one embodiment;

[0017] FIG. **11**A is a block diagram of an example generative language model system suitable for use in implementing at least some embodiments of the present disclosure;

[0018] FIG. **11**B is a block diagram of an example embodiment in which the generative LM includes a transformer encoder-decoder, according to at least one embodiment;

[0019] FIG. **11**C is a block diagram of an example embodiment in which the generative LM **1130** includes a decoder-only transformer architecture, according to at least one embodiment;

[0020] FIG. **12** is a block diagram of an example computing device suitable for use in implementing some embodiments of the present disclosure; and

[0021] FIG. **13** is a block diagram of an example data center suitable for use in implementing some embodiments of the present disclosure.

DETAILED DESCRIPTION

[0022] ASR systems typically analyze a stream of speech data in the form of (suitably preprocessed) time series of spectrograms or frames $F_1, F_2, F_3 \ldots$ of the recorded speech. The spectrograms are processed sequentially, e.g., using a recurrent neural network (RNN) machine learning architecture where a (hidden) state is maintained after processing a given frame F_t and then used to provide global context of the speech as part of processing of the next frame F_{t+1}, being updated again based on that frame F_{t+1}, and so on. Model architectures used in ASR systems include attention-based encoder/decoder models, connectionist temporal classification (CTC) models, transducer models, and/or other models. For example, a transducer model (e.g., as illustrated in FIG. **2**) may include an encoder network that converts, $F_t \rightarrow E_t$, individual frames F_t into encoder feature vectors E_t and a predictor (also sometimes referred to as a decoder) network that generates a current context or state S of the speech. The state S can be based on all recognized units of speech, such as graphemes, subwords, phonemes, etc. A joiner portion of the transducer model can then process the current encoder

vector E_t together with the state of the speech S to generate a prediction vector PE for the next unit of speech being predicted. The prediction vector can be processed by a suitable classifier network or layer, e.g., a softmax layer, to generate probabilities for various vocabulary units of speech being spoken during frame F_t or logarithms of such probabilities (logits). A vocabulary unit having the highest probability can then be selected as the predicted content unit C_u (also referred to as a label herein) spoken during frame F_t. A given frame may contain none (e.g., corresponding to a pause in speech) content units or some content units C_u, e.g., one (e.g., unit "d" in the word "dog"), or more (e.g., units "d" and "o" in the word "dog") content units. A prediction cycle (iteration or instance) is continued for a given frame until a blank (also denoted as Ø herein) unit is predicted. The state of the speech may be updated by the predictor network, e.g., S_{u+1}=Dec(S_u, C_u), when a new content (non-blank) unit is identified.

[0023] Conventional embodiments of decoding algorithms for transducer models (as illustrated schematically in FIG. **3**) include an outer loop that sequentially computes (or selects pre-computed, in the instances of off-line ASR) encoding vectors E_1, E_2, ... E_t and runs both the predictor portion and the joiner portion of the model to predict the new units of speech U. In those instances where the predicted unit U is a non-blank content unit C_u, the state of the speech S_u is updated (e.g., as described above) and the predictor/joiner portion dwell on the same frame F_t (and the corresponding encoding feature vector E_t) repeating prediction of additional content units until a blank is detected. Detection of a predicted blank (U=Ø) concludes predictions for the frame F_t and signals to the model to move to the next frame F_{t+1}. The state of speech remains unchanged S_u since no content unit C_u is predicted. The difference in processing content units C_u and blank predictions Ø presents difficulties for efficient parallel (batch) inference of multiple utterances. In particular, the time (frame) index t may be incremented at different times for different utterances, and the lengths of identified utterances (the number of content units) generally grows at different rates. As a result, calls to execute the predictor network can be scheduled asynchronously. Alternatively, if different utterances are forced to have synchronous prediction processing, processing of at least some utterances can idle. For example, processing of a first utterance for a given time index t can conclude quickly by making a blank prediction Ø, while a second utterance can detect one or more content units for the same t.

[0024] Aspects and embodiments of the present disclosure address these and other challenges of the automated speech processing technology by providing for techniques and systems that implement label-looping processing that optimizes prediction processing. In some embodiments, e.g., as illustrated in FIG. **4**, processing by a speech model may be performed using an outer loop that aggregates non-blank content units C_u identified by an inner loop. More specifically, an individual iteration of the outer loop computes, or updates, based on the most recent identified content speech unit Cu, the state of the speech using a predictor network: S_{u+1}=Dec(S_u,C_u). The state of the speech is not computed (updated) when the identified unit U is a blank (non-content) unit. The inner loop computes (or retrieves from memory) the frame-specific encoder vector E_t and applies the joiner network/classifier to the encoder vector E_t and the updated state of the speech S_{u+1} to predict the next speech unit U. The inner loop further verifies whether the predicted unit U is the blank Ø or a non-blank content speech unit C_{u+1}. In those instances where the inner loop detects a non-blank speech content unit C_{u+1}, the inner loop is interrupted. The outer loop then updates the state of the speech and updates, using the predictor network, the state of the speech. In other instances, where the inner loop detects a blank Ø, the inner loop carries over to its next iteration with the same unmodified state of the speech.

[0025] In the instances of batch processing of multiple speech utterances using parallel processing, the number of calls to the predictor network (for a given number of predicted speech units) is the same across different utterances of the batch, at least as long as the utterance processing has not ended for some of the utterances (so that at least some of the frames remain to be analyzed). Accordingly, the total number of calls to the predictor network is the same as the number of speech units in the longest utterance of the batch.

[0026] The advantages of the disclosed techniques include, but are not limited to, elimination of redundant predictor processing in automatic speech recognition (ASR) applications, speech-to-text (STT) applications, text-to-speech (TTS) applications, speech-to-speech (STS) applications, translation, diarization, and/or other automated speech processing tasks. The predictor network computes an output once per an update of the state of the speech when a new non-blank speech unit is predicted. This leads to optimization of parallel processing of speech recognition tasks. Additionally, the disclosed techniques facilitate efficient execution of ASR algorithms that deploy beam searches. In a beam search, instead of selecting a single most likely speech unit C_u (as done in greedy algorithms), multiple possible speech units may initially be selected: C_u, C'_u, C''_u, etc. Multiple hypotheses are then formed by aggregating various speech units selected for different frames. A hypothesis that maximizes an aggregated probability across the set of frames is then chosen for the predicted speech units. The disclosed techniques, in application to beam search algorithms, have the advantages that various hypotheses have the same length (in the units of speech) thus ensuring that the probabilities associated with different hypotheses can be compared directly, without additional normalization processing.

System Architecture

[0027] FIG. **1**A is a block diagram of an example computer system **100** capable of performing content detection in media items using label-looping prediction, in accordance with at least some embodiments. As depicted in FIG. **1**A, a computer system **100** may include a media content detection (MCD) server **102**, a data store **150**, and a training server **160** connected to a network **140**. Network **140** may be a public network (e.g., the Internet), a private network (e.g., a local area network (LAN), or wide area network (WAN)), a wireless network, a personal area network (PAN), a combination thereof, and/or another network type.

[0028] MCD server **102** may include a desktop computer, a laptop computer, a smartphone, a tablet computer, a server, a wearable device, a VR/AR/MR headset or head-up display, a digital avatar or chatbot kiosk, an in-vehicle infotainment computing device, and/or any suitable computing device capable of performing the techniques described herein. MCD server **102** may be configured to receive media item

101. In some embodiments, media item **101** may include audio data associated with any speech episode involving one or more speakers. Speech episodes may include a public or private conversation, a business meeting, a public or private presentation, an artistic event, a debate, an interaction between a digital agent (e.g., chat bot, digital avatar, etc.) and one or more users, an in-vehicle communication (e.g., between two or more occupants, between an occupant(s) and a chat bot, avatar, or digital assistant of the vehicle), and/or the like. The audio data may be in any suitable format, e.g., WAV, AIFF, MP3, AAC, WMA, or any other compressed or uncompressed audio format.

[0029] In some embodiments, media item **101** may include image(s), video(s) (e.g., temporally, visually, and/or contextually related sequences of images/frames), and or any other data items produced by suitable sensor(s), including but not limited to lidar sensors, radar sensors, infrared camera sensors, temperature sensors, pressure sensors, and/or any other physical or chemical sensors. Media item **101** may be recorded using one or more devices connected to MCD server **102**, retrieved from memory **104** of MCD server **102**, and/or received over any local or network connection (e.g., via network **140**) from an external computing device. In some embodiments, media item **101** may be stored (e.g., together with other data, such as metadata) in data store **150**. Data store **150** may be accessed by MCD server **102** directly or (as shown in FIG. **1**A) via network **140**. Additionally, data store **150** may store training media items **152** for training one or more machine learning models, e.g., content detection model **120**. In some embodiments, content detection model **120** may include an ASR model.

[0030] Data store **150** may include a persistent storage capable of storing audio files and/or other media files as well as metadata for the stored files. Data store **150** may be hosted by one or more storage devices, such as main memory, magnetic or optical storage disks, tapes, or hard drives, network-attached storage (NAS), storage area network (SAN), and so forth. Although depicted as separate from MCD server **102**, in at least some embodiments, data store **150** may be a part of MCD server **102**. In at least some embodiments, data store **150** may be a network-attached file server, while in other embodiments data store **150** may be some other type of persistent storage such as an object-oriented database, a relational database, and so forth, that may be hosted by a server machine or one or more different machines coupled to the MCD server **102** via network **140**.

[0031] MCD server **102** may include a memory **104** (e.g., one or more memory devices or units) communicatively coupled with one or more processing devices, such as one or more graphics processing units (GPU) **110**, one or more central processing units (CPU) **130**, one or more data processing units (DPU), one or more parallel processing units (PPUs), and/or other processing devices (e.g., field-programmable gate arrays (FPGAs), application-specific integrated circuits (ASICs), and/or the like). Memory **104** may store one or more components and models, such as content detection model **120** capable of identifying content units of media item **101**, e.g., converting sounds captured by media item **101** into perceptively distinct speech units (tokens) and various words and sentences made of those speech units. Content detection model **120** may be executed using label-looping prediction **122** that facilitates efficient deployment of content detection model **120**, as disclosed in more detail in conjunction with FIGS. **3**-**5**. Memory **104** may further store a batch processing module **124** that implements parallel processing of batches of media items **101** coordinates calls to the predictor portion.

[0032] Media items **101** (and/or training media items **152**) may be stored in data store **150** in a raw format or in any other suitable representation. In some embodiments, media items **101** (and/or training media items **152**) that include audio data may be stored in the form of spectrograms. For example, a spectrogram of an audio data may be obtained by recording air pressure caused by the speech as a function of time and computing a short-time Fourier transform for overlapping time intervals (frames) of a set duration. This maps the audio signal from the time domain to the frequency domain and generates a spectrogram characterizing the spectral content of the audio data. The amplitude of the audio signal may be represented on a logarithmic (decibel) scale. In some embodiments, the obtained spectrograms may be further converted into mel-spectrograms, by transforming frequency f into a non-linear mel domain, $f \rightarrow m = a \ln(1+f/b)$, to take into account the ability of a human car to better distinguish between equally spaced frequencies (tones) at the lower end of the frequencies of the audible spectrum than at its higher end. In one example, a=1607 and b=700 Hz. Throughout this disclosure, the term "speech spectrogram" may be understood to include Fourier spectrograms or mel-spectrograms, where applicable.

[0033] In at least one embodiment, content detection model **120** may be implemented as a deep learning neural network having multiple levels of linear or non-linear operations. For example, content detection model **120** may include convolutional neural networks, recurrent neural networks, fully-connected neural networks, recurrent neural networks (RNNs), long short-term memory (LSTM) neural networks, neural networks with attention, e.g., transformer neural networks, and/or the like. In at least one embodiment, content detection model **120** may include multiple neurons, an individual neuron receiving its input from other neurons and/or from an external source and producing an output by applying an activation function to the sum of inputs modified by (trainable) weights and a bias value. In at least one embodiment, the neurons may be arranged in layers, including an input layer, one or more hidden layers, and/or an output layer. Neurons from adjacent layers may be connected by weighted edges. In some embodiments, training server **160** may train a number of different models, which may be models that differ by a number of neurons, number of neuron layers, specific neural architecture, and/or the like.

[0034] Initially, parameters (e.g., edge weights and biases) of various network models being trained may be assigned some starting (e.g., random) values. For every training input **164**, training engine **162** may cause content detection model **120** to generate output(s). Training engine **162** may then compare observed output(s) with the desired target output(s) **168**. The resulting error or mismatch, e.g., the difference between the desired target output(s) **168** and the actual output(s) of the neural networks, may be back-propagated through the respective neural networks, and the weights and biases in the neural networks may be adjusted to make the actual outputs closer to the target (ground truth) outputs. This adjustment may be repeated until the output error for a given training input **164** satisfies a predetermined condition (e.g., falls below a predetermined value). Subsequently, a different training input **164** may be selected, a new output generated, and a new series of adjustments implemented,

until the respective neural networks are trained to a target degree of accuracy or until the neural network(s) converges to a limit of its architecture-accuracy.

[0035] Predictive utility of the identified patterns may be subsequently verified using additional training input/target output associations. The trained content detection model **120** and/or other models similarly trained, may be used, during the inference stage, for processing of new (not encountered previously) input speech.

[0036] Content detection model(s) **120** may be trained by training engine **162** hosted by training server **160**, which may be (or include) a desktop computer, a laptop computer, a smartphone, a tablet computer, a server, and/or any suitable computing device capable of performing the techniques described herein. Training of content detection model(s) **120** may be performed using training data (e.g., audio data, video data, and/or other pertinent data) that may be annotated with ground truth **154**, e.g., correct identifications of content portions (e.g., spoken sounds) and/or non-content portions (e.g., noise, pauses, extraneous sounds, etc.) of the training media items **152**.

[0037] During training, predictions of a model **165** being trained (e.g., a content detection model **120**) may be compared with ground truth **154**. More specifically, training engine **162** may cause a model to process training inputs **164**, which may include one or more training media items **152**, and generate training outputs **166**, which represent identifications of content in the corresponding training inputs **164**. During training, training engine **162** may also generate mapping data **167** (e.g., metadata) that associates training inputs **164** with correct target outputs **168**. Target outputs **168** may include ground truth **154** (content identifications) for corresponding training inputs **164**. Training causes the model(s) **165** to identify patterns in training inputs **164** based on desired target outputs **168** and learn to accurately classify input data.

[0038] In some embodiments, training audio data may be used by training engine **162** as training input **164** to train an ASR model to predict likelihoods of various speech units associated with a particular language being spoken during consecutive time intervals t_1, t_2, t_3, etc. In one example embodiment, an output layer of the ASR model may include, for each of N known content (speech) units of the language, a node that outputs a probability p_1, p_2, . . . p_N that the respective speech unit is spoken during a particular time interval or a probability p_0 that a non-content (blank Ø) unit is present in the speech. The probabilities p_0, p_1, . . . p_N may be normalized, $p_0+p_1+p_2+ \ldots +p_N=1$. In some embodiments, the output layer of ASR model may output log-probabilities (logits) $L_k=\ln p_k$.

[0039] Initially, edge parameters (e.g., weights and biases) of the model(s) **165** being trained may be assigned some starting (e.g., random) values. For every training input **164**, training engine **162** may compare training output **166** with the corresponding target output **168**. The resulting error or mismatch, e.g., the difference between the desired target output **168** and the generated training output **166** of model(s) **165**, may be back-propagated through the model(s) **165** and at least some parameters of model(s) **165** may be changed in a way that brings training output **166** closer to target output **168**. Such adjustments may be repeated until the output error for a given training input **164** satisfies a predetermined condition (e.g., falls below a predetermined error). Subsequently, a different training input **164** may be selected, a new training output **166** generated, and a new series of adjustments implemented, until the model is trained to a target degree of precision or until the model converges to a limit of its (architecture-determined) accuracy.

[0040] Training server **160** may train any number of models **165** (e.g., content detection models **120**) using suitable sets of training inputs **164** and target outputs **168**. The trained models **165**-T may be stored in data store **150**, downloaded and deployed on any suitable machine for inference of new data. For example, trained content detection models **120** may be deployed on MCD server **102**.

[0041] In some embodiments, for efficient training, drop-out techniques may be used for at least some of the training epochs, with outputs of at least some neurons removed (e.g., replaced with zero outputs). This forces the remaining neurons to learn how to perform content detection tasks more efficiently and generate more accurate outputs. In the course of training, different neurons (e.g., randomly chosen neurons) may be dropped during processing of different batches of training data, so that all neurons learn to perform tasks more accurately and efficiently.

[0042] FIG. **1**B illustrates an example computing device **103** that supports training or deployment of content detection models that identify content in media items using label-looping prediction, according to at least one embodiment. In at least one embodiment, computing device **103** may be a part of MCD server **102**. In at least one embodiment, computing device **103** may be a part of training server **160**. In at least one embodiment, computing device **103** implements label-looping prediction **122** that supports operations of content detection model **120**. In some embodiments, content prediction model **120** may include an encoder network **230**, a predictor network **240**, a joiner network **250**, and/or other networks, subnetworks, modules, and components that are not explicitly depicted in FIG. **1**B and that may be used to process media item **101**. Batch processing **124** may implement concurrent parallel processing of multiple media items **101**. Operations of content prediction model **120**, label-looping prediction **122**, and/or batch processing **124** may be executed using one or more GPUs **110**, one or more CPUs **130**, one or more parallel processing units (PPUs) or accelerators, such as a deep learning accelerator, data processing units (DPUs), and/or the like. In at least one embodiment, a GPU **110** includes multiple cores **111**, each core being capable of executing multiple threads **112**. Each core may run multiple threads **112** concurrently (e.g., in parallel). In at least one embodiment, threads **112** may have access to registers **113**. Registers **113** may be thread-specific registers with access to a register restricted to a respective thread. Additionally, shared registers **114** may be accessed by one or more (e.g., all) threads of the core. In at least one embodiment, individual cores **111** may include a scheduler **115** to distribute computational tasks and processes among different threads **112** of core **111**. A dispatch unit **116** may implement scheduled tasks on appropriate threads using correct private registers **113** and shared registers **114**. Computing device **103** may include input/output component(s) **134** to facilitate exchange of information with one or more users or developers.

[0043] In at least one embodiment, GPU **110** may have a (high-speed) cache **118**, access to which may be shared by any, some, or all cores **111**. Furthermore, computing device **103** may include a GPU memory **119** where GPU **110** may

store intermediate and/or final results (outputs) of various computations performed by GPU **110**. After completion of a particular task, GPU **110** (or CPU **130**) may move the output to (main) memory **104**. In at least one embodiment, CPU **130** may execute processes that involve serial computational tasks whereas GPU **110** may execute tasks (such as multiplication of inputs of a neural node by weights and addition of biases) that are amenable to parallel processing, e.g., batch processing **124**. In at least one embodiment, label-looping prediction **122** may determine which processes are to be executed on GPU **110** and which processes are to be executed on CPU **130**. In other embodiments, CPU **130** may determine which processes are to be executed on GPU **110** and which processes are to be executed on CPU **130**.

[0044] The systems and methods described herein may be used for a variety of purposes, by way of example and without limitation, for machine control, machine locomotion, machine driving, synthetic data generation, model training, perception, augmented reality, virtual reality, mixed reality, robotics, security and surveillance, simulation and digital twinning, autonomous or semi-autonomous machine applications, deep learning, environment simulation, data center processing, conversational AI, generative AI, light transport simulation (e.g., ray-tracing, path tracing, etc.), collaborative content creation for 3D assets, cloud computing and/or any other suitable applications.

[0045] Disclosed embodiments may be comprised in a variety of different systems such as automotive systems (e.g., a control system for an autonomous or semi-autonomous machine, a perception system for an autonomous or semi-autonomous machine, an in-vehicle infotainment system for an autonomous or semi-autonomous machine), systems implemented using a robot, aerial systems, medical systems, boating systems, smart area monitoring systems, systems for performing deep learning operations, systems for performing simulation operations, systems for performing digital twin operations, systems for performing medical operations, systems for performing factory operations, systems for performing analytics operations, systems implemented using an edge device, systems for generating or presenting at least one of augmented reality content, virtual reality content, mixed reality content, systems incorporating one or more virtual machines (VMs), systems for performing synthetic data generation operations, systems implemented at least partially in a data center, systems for performing conversational AI operations, systems for performing light transport simulation, systems for performing collaborative content creation for 3D assets (e.g., platforms or systems that support universal scene descriptor (USD) data, such as OpenUSD, including but not limited to NVIDIA's OMNIVERSE), systems implementing one or more language models, such as large language models (LLMs), vision language models (VLMs), and/or multi-modal language models that may process text, voice, image, computer aided design (CAD) data, universal scene descriptor (USD) data, 2D and/or 3D graphics data, and/or other data types to generate outputs in one or more formats, systems implemented at least partially using cloud computing resources, systems for performing generative AI operations, and/or other types of systems.

Content Prediction Pipeline

[0046] FIG. **2** illustrates an example architecture and data flow of an audio processing pipeline **200** that implements label-looping prediction for efficient decoding of media content, according to at least one embodiment. In at least one embodiment, audio processing pipeline **200** may be implemented as part of MCD server **102** of FIGS. **1A-1B**, which may be located on a single computing device or on multiple computing devices. Although FIG. **2** illustrates operations of label-looping techniques for ASR processing of audio data, similar techniques may be applied to processing of other speech related tasks, e.g., speech-to-audio conversion, translations of texts from one language to another language, and/or to processing of any other media data that includes temporally (or contextually) related sequences of data units (frames), e.g., videos, streams of sensing data, and/or any other similar data.

[0047] As illustrated in FIG. **2**, audio processing pipeline **200** may receive audio data **204** captured by one or more audio sensors **202**, e.g., microphones. Microphones can include dynamic microphones, condenser microphones, ribbon microphones, unidirectional microphones, omnidirectional microphones, and/or any other types of microphones. In some embodiments, a microphone can be combined with other devices, e.g., computers, phones, speakers, TV screens, and/or the like. The audio data **204** collected by audio sensors **202** may be generated, e.g., spoken, by a single speaker or multiple speakers and may include a single speech episode or multiple speech episodes. Audio sensors **202** may capture not only a speech signal but also background noise, interference signals, e.g., emitted by TV devices, radio devices, alarm devices, and/or any other equipment, or sounds naturally occurring (e.g., sound of wind, water, birds, etc.).

[0048] Audio data **204** collected by audio sensors **202** may undergo speech preprocessing and segmentation **206**. For example, speech preprocessing may include audio filtering, denoising, amplification, dereverberation, and/or any other suitable enhancement. Preprocessing may further include removal of portions of the audio data **204** that do not have a speech content. For example, preprocessing may evaluate energy e(t) associated with the audio data as a function of time and identify regions that have energy less than a certain threshold (e.g., an empirically determined noise threshold). Such identified regions may be removed (trimmed) from the audio data **204** during speech preprocessing. Segmentation may include segmenting the audio data **204** into segments of a predetermined sizes (durations), t, e.g., 0.5-5 sec. Such segments are sometimes referred to as utterances herein. It should be understood that utterances need not correspond to a complete logical unit of speech and may encompass one or more sentences, one or more words, a part of a word (subword), one or more exclamations or other punctuation, filler words, pauses, and/or the like. In some embodiments, the segments may be partially overlapping.

[0049] Individual utterances may be represented by a plurality of audio frames **208**, e.g., M frames over a certain predetermined interval of time. Audio frames **208** may have a duration of 15 msec, 20 msec, 30 msec, and/or some other duration. Audio frames **208** may undergo a suitable frame-to-spectrogram transformation **220**. For example, a spectrogram of a frame may be obtained or generated by performing the discrete Fourier transform of acoustic energy e (t) or air pressure p (t) associated with a specific utterance. The obtained spectrograms $e(f_i)$ may be defined for a number of bands f_1, f_2 . . . f_C, for example, for C=80 bands or C=128 bands, or any other number of bands. In some embodiments,

the bands may be mel-bands and the spectrograms may be mel-spectrograms. Separate frames **222** may be obtained for separate audio frames **208**.

[0050] In some embodiments, one or more content detection models, e.g., a label-looping ASR model **210**, may process a time sequence of frames **222**, e.g., $F_1, \ldots F_{t-1}, F_t, F_{t+1}$, which may include mel-spectrograms of a particular speech episode, and generate likelihoods (e.g., probabilities p_j or log-probabilities $\log p_j$) of various content units **270**, denoted $U_1, U_2, U_3, \ldots$, e.g., graphemes, subwords, phonemes, etc., spoken within consecutive intervals of a set duration t (e.g., 0.25 sec, 0.5 sec, and/or the like). Nested-loop ASR model **210** may generate content units **270** in any suitable language used in training of the model.

[0051] In some embodiments, label-looping ASR model **210** may have an architecture that includes an encoder network **230**, a predictor network **240**, and a joiner network **250**. Predictor network **240** may have an RNN architecture, a long short-term memory (LSTM) architecture, an attention architecture, and/or any other architecture capable of identifying an inter-frame context. For example, RNN or LSTM architecture may store a hidden state that is updated after processing of individual frames and then used in processing of subsequent frames.

[0052] In some embodiments, encoder network **230** converts individual frames F_t into encoder vectors E_t=Enc(F_t). Prediction network **240** may update the current state of the speech based on the most recent identified content unit **270**: S_{u+1}=Dec(S_t, C_u), where index u=1, 2, 3 . . . enumerates identified content units. Joiner network **250** may process an input that includes the encoder vector E_t aggregated with the state vector S_{u+1} and generate a prediction vector P_{u+1} for the next unit of speech. In some embodiments, aggregation of the encoder vector and the state vector may be performed by concatenating or otherwise joining the two vectors. In some embodiments, e.g., in architectures where the encoder vector and the state vector have the same dimensions (the number of vector components), the two vectors may be added together. In other embodiments, e.g., in architectures where the encoder vector and the state vector have different dimensions, at least one of the two vectors may first be projected onto a common space, e.g., a vector (E_t or S_{u+1}) having the larger dimension may be projected onto a space of the other vector, prior to adding the vectors together. In some embodiments, both the encoder vector and the decoder vector may be projected onto a space having a number of dimensions that is different from the dimensionality of each vector.

[0053] Prediction vector P_{u+1} generated by joiner network **250** may be processed by classifier network **260**, such as a softmax classifier. Classifier network **260** may generate probabilities or logits (logarithms of probabilities) for various vocabulary units of speech that may be uttered during frame F_t. A vocabulary unit having the highest probability may be selected as the next predicted content unit C_{u+1}. In some embodiments, the highest probability unit of speech may be a blank Ø unit associated with absence of identifiable speech content. Blank unit filtering **280** may remove blank predictions when forming a representation (e.g., a transcription) of speech content **290** and/or selecting non-blank units to form the state of the speech that is used as an input into prediction network **240**, as described above.

[0054] FIG. **3** illustrates a data flow **300** in a conventional transducer-type ASR system that deploys frame-looping. Operations illustrated with the data flow **300** include setting, at block **310**, the initial frame counter to zero, t=0, the initial content unit counter to zero, u=0, the initial state of speech to a null state S_0=0, and initial content unit to the default beginning-of-sequence value, C_0=BOS. At block **320**, the current frame is processed by the encoder network to produce the corresponding encoder vector, E_t=Enc(F_t). At block **330**, the predictor network generates the state vector, S_{u+1}=Dec(S_u,C_u) by processing the state of speech that includes previously predicted content units of speech. At block **340**, At block **340**, the updated state vector and the encoder vector are processed by the joiner network to obtain the identified unit U. As described in conjunction with FIG. **2**, block **340** can be performed by first computing the prediction vector P_{u+1}=Joint (E_t,S_{u+1}), processing the prediction vector using a classifier network, $\{L_i\}$=Softmax(P_{u+1}), to generate a set of likelihoods $\{L_i\}$ (probabilities, logits) for the prediction vector P_{u+1} that corresponds to a set of vocabulary units $\{V_i\}$ or to the blank Ø unit, and then selecting the unit with the highest likelihood: U=max($\{L_i\}$, $L_\emptyset$). The decision-making block **350** verifies whether the identified unit U is the blank Ø (non-content) unit or a content unit. If the identified unit is blank Ø, the times index is advanced, at block **360**, and the next frame processing is started with the next outer loop iteration. If the identified unit U is non-blank, the identified unit is stored as the next content unit U→C_{u+1}, at block **370**, which also updates the content unit counter, and the next iteration of the inner loop commences, including updating the state of speech at block **330**, and/so on. The processing continues until all frames are processed.

[0055] FIG. **4** illustrates an example data flow **400** in a transducer-type ASR system that deploys label-looping prediction for efficient decoding of media content, according to at least one embodiment. Operations illustrated in FIG. **4** may be performed by MCD server **102** (with reference to FIGS. **1**A and **1**B) and may begin with obtaining audio data, e.g., a recording of a speech episode that includes one or more speakers. Speakers may be human or artificial, e.g., chatbots, digital avatars, TTS models, and/or the like. The audio data may include streaming data (e.g., live speech transcription or translation) or data previously recorded and stored. In some embodiments, operations illustrated in FIG. **4** may be performed in conjunction with non-audio data (e.g., sensor data) or data that includes both audio data and non-audio data (e.g., video data), and/or the like.

[0056] As illustrated in FIG. **4**, data flow **400** may include setting, at block **410**, the initial frame counter to zero, t=0, the initial content unit counter to zero, u=0, the initial state of speech to the null state S_0=0, and initial content unit to the default beginning-of-sequence value, C_0=BOS. At block **420**, the predictor network may generate the state vector, S_{u+1}=Dec(S_u, C_u) by processing the state of speech that includes previously predicted content units of speech. Operations of block **420** may be performed as part of the outer loop of the label-looping prediction algorithm.

[0057] At block **430**, operations of the inner loop may begin with computing the encoder vector by processing the current frame using the encoder network, E_t=Enc(F_t). At block **440**, the updated state vector and encoder vector may be processed by the joiner network to obtain the identified content unit. Block **440** may be performed by first computing the prediction vector P_{u+1}=Joint(E_t, S_{u+1}), processing the prediction vector using a (e.g., softmax) classifier network, $\{L_i\}$=Softmax(P_{u+1}), to generate a set of likelihoods $\{L_i\}$

(probabilities, logits) based on the prediction vector P_{u+1}, the set of likelihoods indicating which of the vocabulary units $\{V_i\}$ or to the blank Ø unit is likely being captured by frame F_t, and then selecting the unit with the highest likelihood: $U_t=\max(\{L_i\}, L_\emptyset)$.

[0058] The decision-making block **450** verifies whether the identified unit U is the blank Ø (non-content) unit or a content unit. If the identified unit is blank Ø, the current state of speech is maintained, the frame index is advanced, at block **460**, and a new frame processing commences, at the next iteration of the inner loop.

[0059] If the identified unit U is non-blank, the identified unit is stored as the next content unit, $U \rightarrow C_{u+1}$, at block **470**, which also updates the content unit counter, and the next iteration of the outer loop begins, including updating the state of speech, at block **420**. The next iteration of the outer loop is then performed and a new sequence of one or more inner loops (for one or more frames F_{t+1}, F_{t+2}, . . .) is then performed until the next non-blank content unit is identified.

[0060] FIG. **5** illustrates an example data flow **500** in a transducer-type ASR system that deploys label-looping prediction for batch processing of media content, according to at least one embodiment. In parallel batch processing of multiple speech utterances, the number of calls to the predictor network (for a given number N of predicted speech units) is the same across different utterances of the batch until the processing ends for at least some of the utterance (s). Since the states of different utterances j are updated concurrently and the total number of calls to the predictor network is the same per each iteration of the outer loop, the states can be joined into a state tensor $\{S_{j,u}\}$, with j tracking a timestamp t_j of a currently processed frame F_{t_j} of jth utterance. At block **510**, the states of all utterances may be initialized to the null state, $S_{j,0}=0$, the initial content units may be set to the default beginning-of-sequence value, $C_{j,0}$=BOS. and the initial (individual) frame counters t; and the (common) unit counter u may be set to zero for all utterances of the batch, $\{t_j\}=0$, u=0. At block **520**, the predictor network may (synchronously) generate the state vectors, $S_{j,u+1}=\mathrm{Dec}(S_{j,u}, C_{j,u})$ by processing the states of speech that include previously predicted content units $C_{j,u}$. Operations of the outer loop may be performed using a joint vector of frame counters $\{t_j\}$ and the state tensor $\{S_j\}$. Operations of the inner loops, including block **530** (computation of the encoder vectors), block **540** (prediction of speech units), and block **550** (non-blank unit detection), and block **560** (frame counter update) may be performed until new units of speech have been identified for all individual utterances, at which point the identified units, at block **570**, are stored as $C_{j,u+1}$ and the common unit counter is updated, followed by an update, at block **520**, to the states of individual utterances (as part of the next iteration of the outer loop). In one example, computations of block **520** may be performed using NVIDIA® CUDA® tensors. The number of iterations of the outer loop are, therefore, the same for all utterances in the batch. The number of iterations in the inner loop (performed for each instance of the outer loop) is, generally, utterance-specific, as different utterances have different numbers of units for individual frames. If the inner loop for jth utterance detects the blank unit Ø, block **560** advances individual timestamps of the utterances. The number of remaining frames in each utterance may be tracked, with further processing of an utterance stopped when no frame in the utterance remain to be decoded.

[0061] The following pseudo-code corresponds to one example algorithm that implements the data flow **500** of FIG. **5**:

```
1:      input: acoustic input x1, x2, ..., xB, input length
2:      encs = encoder(x) # output dim is [B, T, dim]
3:      hyps, state = [[ ] * B], [predictor.init_state( ) * B]
4:      b2active, b2time = [True * B], [0 * B]
5:      while b2active.any( ) do
6:          decs, states = decoder(state, predictions)
7:          token_probs = joiner(encs[b2time], decs)
8:          predictions = argmax(token_probs)
9:          blank mask = (predictions == Ø)
10:         b2time[blank mask] += 1
11:         b2active = b2time ≤ input length
12:         while (blank mask AND b2active).any( ) do
13:             token_probs = joiner(encs[b2time], decs)
14:             extra predictions = argmax(token_probs)
15:             extra blank mask = (predictions == Ø)
16:             predictions[blank mask] = \
                        extra predictions[blank mask]
17:             blank mask = blank mask AND extra blank mask
18:             b2time[blank mask] += 1
19:             b2active = b2time ≤ input length
20:         hyps.append(predictions)
21:     return units
```

[0062] In some embodiments, the ASR system deploying the label-looping prediction techniques may deploy token-and-duration transducer (TDT) model. A TDT model may have an architecture that is similar to the architecture illustrated in FIG. **2** and FIGS. **4-5** with the joiner network **250** having a second output, which may be processed by a second classifier network that predicts a duration t for various content units C_u. In such systems, operations of block **420**, which increments the timestamp of a frame, may advance the timestamp by the predicted duration, t=t+t. Correspondingly, the above pseudo-code is still applicable in TDT embodiments, with one difference being that updating b2time values at lines 10 and 18 is performed according to the duration prediction made by the second classifier network.

[0063] FIG. **6** is a flow diagram of a method **600** of automatic content recognition that deploys label-looping prediction, according to at least one embodiment. Method **600** may be performed using one or more processing units (e.g., CPUs, GPUs, accelerators, PPUs, DPUs, etc.), which may include (or communicate with) one or more memory devices. In at least one embodiment, method **600** may be performed using processing units of MCD server **102**. In at least one embodiment, processing units performing method **600** may be executing instructions stored on non-transient computer-readable storage media. In at least one embodiment, method **600** may be performed using multiple processing threads (e.g., CPU threads and/or GPU threads), individual threads executing one or more individual functions, routines, subroutines, or operations of the method. In at least one embodiment, processing threads implementing method **600** may be synchronized (e.g., using semaphores, critical sections, and/or other thread synchronization mechanisms). Alternatively, processing threads implementing method **600** may be executed asynchronously with respect to each other. Some operations of method **600** may be performed in a different order compared with the order shown in FIG. **6**. Some operations of method **600** may be performed

concurrently with other operations. In at least one embodiment, one or more operations shown in FIG. **6** may not always be performed.

[0064] Method **600** may involve speech utterances produced by people in any possible context, e.g., a conversation, a public speech, a public event, a business meeting, a conference, a street encounter, an interaction in a game, an interaction with a chat bot or a digital avatar, an interaction with an in-vehicle infotainment system, and/or the like. "Speech," as used in the context of method **600** may also be understood as also including non-human sounds, e.g., sounds of animals. "Speech," as used in the context of method **600** may also include sounds produced by non-living entities, including natural forces, such as wind, sea, ocean, thunderstorms, and various other atmospheric or naval phenomena, as well as robotic speech, synthesized or computer-generated speech, and so on. One or more operations of method **600** may be performed by MCD server **102** of FIG. **1** and/or FIG. **2**.

[0065] Method **600** may include processing, using a content detection model (e.g., content detection model **120** of FIGS. **1**A and **1**B, label-looping ASR model **210** of FIG. **2**, and/or the like), a media item (e.g., media item **101** in FIGS. **1**A and **1**B). The media item may include an audio item (e.g., audio data **204** in FIG. **2**), a video item, a streaming sensor data item, and/or the like. In some embodiments, the media item may include one or more speech utterances. The media item may be represented via a plurality of frames, e.g., spectrograms, video frames, sensing data frames, and/or the like. Method **600** may be performed using multiple iterations of an outer processing loop, each outer loop iteration identifying a content unit (e.g., grapheme, subword, phoneme, etc.) of the media item. An individual iteration of the outer loop may include one iteration (if a content unit is identified during the next processed frame of the media item) or multiple iterations of the inner processing loop (if at least one blank unit Ø is identified prior to a content unit). In some embodiments, prior to a first iteration of the plurality of iterations of the outer processing loop, method **600** may include setting a state of the media item to a default beginning-of-sequence (BOS) state.

[0066] At block **610**, method **600** may include performing a plurality of iterations of the outer processing loop to identify a plurality of content units (CUs) of the media item. As illustrated with the top callout portion **611**, an individual iteration of the plurality of iterations of the outer processing loop may include updating, using a first neural network (NN) and an identified non-blank CU, a state of the media item. In some embodiments, method **600** may include, at block **612**, processing, using the first NN, the state of the media item and the identified non-blank CU. In some embodiments, the first NN may include a predictor NN (e.g., predictor network **240** of FIG. **2**).

[0067] At block **620**, method **600** may include performing one or more iterations of the inner processing loop. As illustrated with the bottom callout portion **621**, an individual iteration of the one or more iterations of the inner processing loop may include processing, using a second NN, the state of the media item and an individual frame of the plurality of frames to predict a CU associated with the individual frame. In some embodiments, the second NN may include a joiner NN (e.g., joiner network **250** of FIG. **2**). The one or more iterations of the inner processing loop may be performed until the predicted CU corresponds to a non-blank CU. More specifically, at block **622**, method **600** may include processing, using an encoder NN (e.g., encoder network **230** of FIG. **2**), the individual frame to obtain an encoder vector. At block **624**, method **600** may include processing, using the joiner NN, the state (e.g., represented via decoder vector Dt) and the encoder vector (e.g., encoder vector E_t in FIG. **2**) to generate a prediction vector (e.g., prediction vector P_t in FIG. **2**) for the individual frame. At block **626**, method **600** may include generating, using a classifier NN (e.g., classifier network **260** of FIG. **2**) and the prediction vector, a plurality of probabilities that the individual frame is associated with (at least) a plurality of vocabulary CUs or a blank CU. At block **628**, method **600** may continue with maintaining, responsive to determining that the predicted CU corresponds to the blank CU, the state of the media item (e.g., as illustrated with block **460** of FIG. **4**). The next iteration of the plurality of iterations of the outer processing loop may performed responsive to identification of the non-blank CU (which corresponds to the interruption of the inner processing loop).

[0068] At block **630**, method **600** may include generating, using the identified plurality of CUs, a representation of the media item. In some embodiments, the representation of the media item may include a transcription of the speech utterance.

[0069] In some embodiments, method **600** may be performed in parallel for multiple media items. For example, the plurality of iterations of the outer processing loop to identify the plurality of CUs of the media item may be performed in parallel to a second (third, etc.) plurality of iterations of the outer processing loop performed to identify a second (third, etc.) plurality of CUs of a second (third, etc.) media item. The plurality of iterations and the second (third, etc.) plurality of iterations may include an equal number of calls to the first NN to identify an equal number of CUs), e.g., as disclosed in conjunction with FIG. **5**.

Inference and Training Logic

[0070] FIG. **7**A illustrates inference and/or training logic **715** used to perform inferencing and/or training operations associated with one or more embodiments.

[0071] In at least one embodiment, inference and/or training logic **715** may include, without limitation, code and/or data storage **701** to store forward and/or output weight and/or input/output data, and/or other parameters to configure neurons or layers of a neural network trained and/or used for inferencing in aspects of one or more embodiments. In at least one embodiment, training logic **715** may include, or be coupled to code and/or data storage **701** to store graph code or other software to control timing and/or order, in which weight and/or other parameter information is to be loaded to configure, logic, including integer and/or floating point units (collectively, arithmetic logic units (ALUs) or simply circuits). In at least one embodiment, code, such as graph code, loads weight or other parameter information into processor ALUs based on an architecture of a neural network to which such code corresponds. In at least one embodiment, code and/or data storage **701** stores weight parameters and/or input/output data of each layer of a neural network trained or used in conjunction with one or more embodiments during forward propagation of input/output data and/or weight parameters during training and/or inferencing using aspects of one or more embodiments. In at least one embodiment, any portion of code and/or data storage

701 may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory.

[0072] In at least one embodiment, any portion of code and/or data storage 701 may be internal or external to one or more processors or other hardware logic devices or circuits. In at least one embodiment, code and/or code and/or data storage 701 may be cache memory, dynamic randomly addressable memory ("DRAM"), static randomly addressable memory ("SRAM"), non-volatile memory (e.g., flash memory), or other storage. In at least one embodiment, a choice of whether code and/or code and/or data storage 701 is internal or external to a processor, for example, or comprising DRAM, SRAM, flash or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0073] In at least one embodiment, inference and/or training logic 715 may include, without limitation, a code and/or data storage 705 to store backward and/or output weight and/or input/output data corresponding to neurons or layers of a neural network trained and/or used for inferencing in aspects of one or more embodiments. In at least one embodiment, code and/or data storage 705 stores weight parameters and/or input/output data of each layer of a neural network trained or used in conjunction with one or more embodiments during backward propagation of input/output data and/or weight parameters during training and/or inferencing using aspects of one or more embodiments. In at least one embodiment, training logic 715 may include, or be coupled to code and/or data storage 705 to store graph code or other software to control timing and/or order, in which weight and/or other parameter information is to be loaded to configure, logic, including integer and/or floating point units (collectively, arithmetic logic units (ALUs).

[0074] In at least one embodiment, code, such as graph code, causes the loading of weight or other parameter information into processor ALUs based on an architecture of a neural network to which such code corresponds. In at least one embodiment, any portion of code and/or data storage 705 may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory. In at least one embodiment, any portion of code and/or data storage 705 may be internal or external to one or more processors or other hardware logic devices or circuits. In at least one embodiment, code and/or data storage 705 may be cache memory, DRAM, SRAM, non-volatile memory (e.g., flash memory), or other storage. In at least one embodiment, a choice of whether code and/or data storage 705 is internal or external to a processor, for example, or comprising DRAM, SRAM, flash memory or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0075] In at least one embodiment, code and/or data storage 701 and code and/or data storage 705 may be separate storage structures. In at least one embodiment, code and/or data storage 701 and code and/or data storage 705 may be a combined storage structure. In at least one embodiment, code and/or data storage 701 and code and/or data storage 705 may be partially combined and partially separate. In at least one embodiment, any portion of code and/or data storage 701 and code and/or data storage 705 may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory.

[0076] In at least one embodiment, inference and/or training logic 715 may include, without limitation, one or more arithmetic logic unit(s) ("ALU(s)") 710, including integer and/or floating point units, to perform logical and/or mathematical operations based, at least in part on, or indicated by, training and/or inference code (e.g., graph code), a result of which may produce activations (e.g., output values from layers or neurons within a neural network) stored in an activation storage 720 that are functions of input/output and/or weight parameter data stored in code and/or data storage 701 and/or code and/or data storage 705. In at least one embodiment, activations stored in activation storage 720 are generated according to linear algebraic and or matrix-based mathematics performed by ALU(s) 710 in response to performing instructions or other code, wherein weight values stored in code and/or data storage 705 and/or data storage 701 are used as operands along with other values, such as bias values, gradient information, momentum values, or other parameters or hyperparameters, any or all of which may be stored in code and/or data storage 705 or code and/or data storage 701 or another storage on or off-chip.

[0077] In at least one embodiment, ALU(s) 710 are included within one or more processors or other hardware logic devices or circuits, whereas in another embodiment, ALU(s) 710 may be external to a processor or other hardware logic device or circuit that uses them (e.g., a co-processor). In at least one embodiment, ALU(s) 710 may be included within a processor's execution units or otherwise within a bank of ALUs accessible by a processor's execution units either within same processor or distributed between different processors of different types (e.g., central processing units, graphics processing units, fixed function units, etc.). In at least one embodiment, code and/or data storage 701, code and/or data storage 705, and activation storage 720 may share a processor or other hardware logic device or circuit, whereas in another embodiment, they may be in different processors or other hardware logic devices or circuits, or some combination of same and different processors or other hardware logic devices or circuits. In at least one embodiment, any portion of activation storage 720 may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory. Furthermore, inferencing and/or training code may be stored with other code accessible to a processor or other hardware logic or circuit and fetched and/or processed using a processor's fetch, decode, scheduling, execution, retirement and/or other logical circuits.

[0078] In at least one embodiment, activation storage 720 may be cache memory, DRAM, SRAM, non-volatile memory (e.g., flash memory), or other storage. In at least one embodiment, activation storage 720 may be completely or partially within or external to one or more processors or other logical circuits. In at least one embodiment, a choice of whether activation storage 720 is internal or external to a processor, for example, or comprising DRAM, SRAM, flash memory or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch

size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0079] In at least one embodiment, inference and/or training logic **715** illustrated in FIG. **7**A may be used in conjunction with an application-specific integrated circuit ("ASIC"), such as a TensorFlow® Processing Unit from Google, an inference processing unit (IPU) from Graphcore™, or a Nervana® (e.g., "Lake Crest") processor from Intel Corp. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. **7**A may be used in conjunction with central processing unit ("CPU") hardware, graphics processing unit ("GPU") hardware or other hardware, such as field programmable gate arrays ("FPGAs").

[0080] FIG. **7**B illustrates inference and/or training logic **715**, according to at least one embodiment. In at least one embodiment, inference and/or training logic **715** may include, without limitation, hardware logic in which computational resources are dedicated or otherwise exclusively used in conjunction with weight values or other information corresponding to one or more layers of neurons within a neural network. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. **7**B may be used in conjunction with an application-specific integrated circuit (ASIC), such as TensorFlow® Processing Unit from Google, an inference processing unit (IPU) from Graphcore™, or a Nervana® (e.g., "Lake Crest") processor from Intel Corp. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. **7**B may be used in conjunction with central processing unit (CPU) hardware, graphics processing unit (GPU) hardware or other hardware, such as field programmable gate arrays (FPGAs). In at least one embodiment, inference and/or training logic **715** includes, without limitation, code and/or data storage **701** and code and/or data storage **705**, which may be used to store code (e.g., graph code), weight values and/or other information, including bias values, gradient information, momentum values, and/or other parameter or hyperparameter information. In at least one embodiment illustrated in FIG. **7**B, each of code and/or data storage **701** and code and/or data storage **705** is associated with a dedicated computational resource, such as computational hardware **702** and computational hardware **706**, respectively. In at least one embodiment, each of computational hardware **702** and computational hardware **706** comprises one or more ALUs that perform mathematical functions, such as linear algebraic functions, only on information stored in code and/or data storage **701** and code and/or data storage **705**, respectively, result of which is stored in activation storage **720**.

[0081] In at least one embodiment, each of code and/or data storage **701** and **705** and corresponding computational hardware **702** and **706**, respectively, correspond to different layers of a neural network, such that resulting activation from one storage/computational pair **701**/**702** of code and/or data storage **701** and computational hardware **702** is provided as an input to a next storage/computational pair **705**/**706** of code and/or data storage **705** and computational hardware **706**, in order to mirror a conceptual organization of a neural network. In at least one embodiment, each of storage/computational pairs **701**/**702** and **705**/**706** may correspond to more than one neural network layer. In at least one embodiment, additional storage/computation pairs (not shown) subsequent to or in parallel with storage/computation pairs **701**/**702** and **705**/**706** may be included in inference and/or training logic **715**.

Neural Network Training and Deployment

[0082] FIG. **8** illustrates training and deployment of a deep neural network, according to at least one embodiment. In at least one embodiment, untrained neural network **806** is trained using a training dataset **802**. In at least one embodiment, training framework **804** is a PyTorch framework, whereas in other embodiments, training framework **804** is a TensorFlow, Boost, Caffe, Microsoft Cognitive Toolkit/CNTK, MXNet, Chainer, Keras, Deeplearning4j, or other training framework. In at least one embodiment, training framework **804** trains an untrained neural network **806** and enables it to be trained using processing resources described herein to generate a trained neural network **808**. In at least one embodiment, weights may be chosen randomly or by pre-training using a deep belief network. In at least one embodiment, training may be performed in either a supervised, partially supervised, or unsupervised manner.

[0083] In at least one embodiment, untrained neural network **806** is trained using supervised learning, wherein training dataset **802** includes an input paired with a desired output for an input, or where training dataset **802** includes input having a known output and an output of neural network **806** is manually graded. In at least one embodiment, untrained neural network **806** is trained in a supervised manner and processes inputs from training dataset **802** and compares resulting outputs against a set of expected or desired outputs. In at least one embodiment, errors are then propagated back through untrained neural network **806**. In at least one embodiment, training framework **804** adjusts weights that control untrained neural network **806**. In at least one embodiment, training framework **804** includes tools to monitor how well untrained neural network **806** is converging towards a model, such as trained neural network **808**, suitable to generating correct answers, such as in result **814**, based on input data such as a new dataset **812**. In at least one embodiment, training framework **804** trains untrained neural network **806** repeatedly while adjusting weights to refine an output of untrained neural network **806** using a loss function and adjustment algorithm, such as stochastic gradient descent. In at least one embodiment, training framework **804** trains untrained neural network **806** until untrained neural network **806** achieves a desired accuracy. In at least one embodiment, trained neural network **808** can then be deployed to implement any number of machine learning operations.

[0084] In at least one embodiment, untrained neural network **806** is trained using unsupervised learning, whereas untrained neural network **806** attempts to train itself using unlabeled data. In at least one embodiment, unsupervised learning training dataset **802** will include input data without any associated output data or "ground truth" data. In at least one embodiment, untrained neural network **806** can learn groupings within training dataset **802** and can determine how individual inputs are related to untrained dataset **802**. In at least one embodiment, unsupervised training can be used to generate a self-organizing map in trained neural network **808** capable of performing operations useful in reducing dimensionality of new dataset **812**. In at least one embodiment, unsupervised training can also be used to perform

anomaly detection, which allows identification of data points in new dataset **812** that deviate from normal patterns of new dataset **812**.

[0085] In at least one embodiment, semi-supervised learning may be used, which is a technique in which in training dataset **802** includes a mix of labeled and unlabeled data. In at least one embodiment, training framework **804** may be used to perform incremental learning, such as through transferred learning techniques. In at least one embodiment, incremental learning enables trained neural network **808** to adapt to new dataset **812** without forgetting knowledge instilled within trained neural network **808** during initial training.

[0086] With reference to FIG. **9**, FIG. **9** is an example data flow diagram for a process **900** of generating and deploying a processing and inferencing pipeline, according to at least one embodiment. In at least one embodiment, process **900** may be deployed to perform game name recognition analysis and inferencing on user feedback data at one or more facilities **902**, such as a data center.

[0087] In at least one embodiment, process **900** may be executed within a training system **904** and/or a deployment system **906**. In at least one embodiment, training system **904** may be used to perform training, deployment, and embodiment of machine learning models (e.g., neural networks, object detection algorithms, computer vision algorithms, etc.) for use in deployment system **906**. In at least one embodiment, deployment system **906** may be configured to offload processing and compute resources among a distributed computing environment to reduce infrastructure requirements at facility **902**. In at least one embodiment, deployment system **906** may provide a streamlined platform for selecting, customizing, and implementing virtual instruments for use with computing devices at facility **902**. In at least one embodiment, virtual instruments may include software-defined applications for performing one or more processing operations with respect to feedback data. In at least one embodiment, one or more applications in a pipeline may use or call upon services (e.g., inference, visualization, compute, AI, etc.) of deployment system **906** during execution of applications.

[0088] In at least one embodiment, some applications used in advanced processing and inferencing pipelines may use machine learning models or other AI to perform one or more processing steps. In at least one embodiment, machine learning models may be trained at facility **902** using feedback data **908** (such as imaging data) stored at facility **902** or feedback data **908** from another facility or facilities, or a combination thereof. In at least one embodiment, training system **904** may be used to provide applications, services, and/or other resources for generating working, deployable machine learning models for deployment system **906**.

[0089] In at least one embodiment, a model registry **924** may be backed by object storage that may support versioning and object metadata. In at least one embodiment, object storage may be accessible through, for example, a cloud storage (e.g., a cloud **1026** of FIG. **10**) compatible application programming interface (API) from within a cloud platform. In at least one embodiment, machine learning models within model registry **924** may be uploaded, listed, modified, or deleted by developers or partners of a system interacting with an API. In at least one embodiment, an API may provide access to methods that allow users with appropriate credentials to associate models with applications, such that models may be executed as part of execution of containerized instantiations of applications.

[0090] In at least one embodiment, a training pipeline **1004** (FIG. **10**) may include a scenario where facility **902** is training their own machine learning model, or has an existing machine learning model that needs to be optimized or updated. In at least one embodiment, feedback data **908** may be received from various channels, such as forums, web forms, or the like. In at least one embodiment, once feedback data **908** is received, AI-assisted annotation **910** may be used to aid in generating annotations corresponding to feedback data **908** to be used as ground truth data for a machine learning model. In at least one embodiment, AI-assisted annotation **910** may include one or more machine learning models (e.g., convolutional neural networks (CNNs)) that may be trained to generate annotations corresponding to certain types of feedback data **908** (e.g., from certain devices) and/or certain types of anomalies in feedback data **908**. In at least one embodiment, AI-assisted annotations **910** may then be used directly, or may be adjusted or fine-tuned using an annotation tool, to generate ground truth data. In at least one embodiment, in some examples, labeled data **912** may be used as ground truth data for training a machine learning model. In at least one embodiment, AI-assisted annotations **910**, labeled data **912**, or a combination thereof may be used as ground truth data for training a machine learning model, e.g., via model training **914** in FIGS. **9-10**. In at least one embodiment, a trained machine learning model may be referred to as an output model **916**, and may be used by deployment system **906**, as described herein.

[0091] In at least one embodiment, training pipeline **1004** (FIG. **10**) may include a scenario where facility **902** needs a machine learning model for use in performing one or more processing tasks for one or more applications in deployment system **906**, but facility **902** may not currently have such a machine learning model (or may not have a model that is optimized, efficient, or effective for such purposes). In at least one embodiment, an existing machine learning model may be selected from model registry **924**. In at least one embodiment, model registry **924** may include machine learning models trained to perform a variety of different inference tasks on imaging data. In at least one embodiment, machine learning models in model registry **924** may have been trained on imaging data from different facilities than facility **902** (e.g., facilities that are remotely located). In at least one embodiment, machine learning models may have been trained on imaging data from one location, two locations, or any number of locations. In at least one embodiment, when being trained on imaging data, which may be a form of feedback data **908**, from a specific location, training may take place at that location, or at least in a manner that protects confidentiality of imaging data or restricts imaging data from being transferred off-premises (e.g., to comply with HIPAA regulations, privacy regulations, etc.). In at least one embodiment, once a model is trained—or partially trained—at one location, a machine learning model may be added to model registry **924**. In at least one embodiment, a machine learning model may then be retrained, or updated, at any number of other facilities, and a retrained or updated model may be made available in model registry **924**. In at least one embodiment, a machine learning model may then be selected from model registry **924**—and referred to as output model **916**—and may be used in deployment system

906 to perform one or more processing tasks for one or more applications of a deployment system.

[0092] In at least one embodiment, training pipeline **1004** (FIG. **10**) may be used in a scenario that includes facility **902** requiring a machine learning model for use in performing one or more processing tasks for one or more applications in deployment system **906**, but facility **902** may not currently have such a machine learning model (or may not have a model that is optimized, efficient, or effective for such purposes). In at least one embodiment, a machine learning model selected from model registry **924** might not be fine-tuned or optimized for feedback data **908** generated at facility **902** because of differences in populations, genetic variations, robustness of training data used to train a machine learning model, diversity in anomalies of training data, and/or other issues with training data. In at least one embodiment, AI-assisted annotation **910** may be used to aid in generating annotations corresponding to feedback data **908** to be used as ground truth data for retraining or updating a machine learning model. In at least one embodiment, labeled data **912** may be used as ground truth data for training a machine learning model. In at least one embodiment, retraining or updating a machine learning model may be referred to as model training **914**. In at least one embodiment, model training **914**—e.g., AI-assisted annotations **910**, labeled data **912**, or a combination thereof—may be used as ground truth data for retraining or updating a machine learning model.

[0093] In at least one embodiment, deployment system **906** may include software **918**, services **920**, hardware **922**, and/or other components, features, and functionality. In at least one embodiment, deployment system **906** may include a software "stack," such that software **918** may be built on top of services **920** and may use services **920** to perform some or all of processing tasks, and services **920** and software **918** may be built on top of hardware **922** and use hardware **922** to execute processing, storage, and/or other compute tasks of deployment system **906**.

[0094] In at least one embodiment, software **918** may include any number of different containers, where each container may execute an instantiation of an application. In at least one embodiment, each application may perform one or more processing tasks in an advanced processing and inferencing pipeline (e.g., inferencing, object detection, feature detection, segmentation, image enhancement, calibration, etc.). In at least one embodiment, for each type of computing device there may be any number of containers that may perform a data processing task with respect to feedback data **908** (or other data types, such as those described herein). In at least one embodiment, an advanced processing and inferencing pipeline may be defined based on selections of different containers that are desired or required for processing feedback data **908**, in addition to containers that receive and configure imaging data for use by each container and/or for use by facility **902** after processing through a pipeline (e.g., to convert outputs back to a usable data type for storage and display at facility **902**). In at least one embodiment, a combination of containers within software **918** (e.g., that make up a pipeline) may be referred to as a virtual instrument (as described in more detail herein), and a virtual instrument may leverage services **920** and hardware **922** to execute some or all processing tasks of applications instantiated in containers.

[0095] In at least one embodiment, data may undergo pre-processing as part of data processing pipeline to prepare data for processing by one or more applications. In at least one embodiment, post-processing may be performed on an output of one or more inferencing tasks or other processing tasks of a pipeline to prepare an output data for a next application and/or to prepare output data for transmission and/or use by a user (e.g., as a response to an inference request). In at least one embodiment, inferencing tasks may be performed by one or more machine learning models, such as trained or deployed neural networks, which may include output models **916** of training system **904**.

[0096] In at least one embodiment, tasks of data processing pipeline may be encapsulated in one or more container(s) that each represent a discrete, fully functional instantiation of an application and virtualized computing environment that is able to reference machine learning models. In at least one embodiment, containers or applications may be published into a private (e.g., limited access) area of a container registry (described in more detail herein), and trained or deployed models may be stored in model registry **924** and associated with one or more applications. In at least one embodiment, images of applications (e.g., container images) may be available in a container registry, and once selected by a user from a container registry for deployment in a pipeline, an image may be used to generate a container for an instantiation of an application for use by a user system.

[0097] In at least one embodiment, developers may develop, publish, and store applications (e.g., as containers) for performing processing and/or inferencing on supplied data. In at least one embodiment, development, publishing, and/or storing may be performed using a software development kit (SDK) associated with a system (e.g., to ensure that an application and/or container developed is compliant with or compatible with a system). In at least one embodiment, an application that is developed may be tested locally (e.g., at a first facility, on data from a first facility) with an SDK which may support at least some of services **920** as a system (e.g., system **1000** of FIG. **10**). In at least one embodiment, once validated by system **1000** (e.g., for accuracy, etc.), an application may be available in a container registry for selection and/or embodiment by a user (e.g., a hospital, clinic, lab, healthcare provider, etc.) to perform one or more processing tasks with respect to data at a facility (e.g., a second facility) of a user.

[0098] In at least one embodiment, developers may then share applications or containers through a network for access and use by users of a system (e.g., system **1000** of FIG. **10**). In at least one embodiment, completed and validated applications or containers may be stored in a container registry and associated machine learning models may be stored in model registry **924**. In at least one embodiment, a requesting entity that provides an inference or image processing request may browse a container registry and/or model registry **924** for an application, container, dataset, machine learning model, etc., select a desired combination of elements for inclusion in data processing pipeline, and submit a processing request. In at least one embodiment, a request may include input data that is necessary to perform a request, and/or may include a selection of application(s) and/or machine learning models to be executed in processing a request. In at least one embodiment, a request may then be passed to one or more components of deployment system

906 (e.g., a cloud) to perform processing of a data processing pipeline. In at least one embodiment, processing by deployment system **906** may include referencing selected elements (e.g., applications, containers, models, etc.) from a container registry and/or model registry **924**. In at least one embodiment, once results are generated by a pipeline, results may be returned to a user for reference (e.g., for viewing in a viewing application suite executing on a local, on-premises workstation or terminal).

[0099] In at least one embodiment, to aid in processing or execution of applications or containers in pipelines, services **920** may be leveraged. In at least one embodiment, services **920** may include compute services, collaborative content creation services, simulation services, artificial intelligence (AI) services, visualization services, and/or other service types. In at least one embodiment, services **920** may provide functionality that is common to one or more applications in software **918**, so functionality may be abstracted to a service that may be called upon or leveraged by applications. In at least one embodiment, functionality provided by services **920** may run dynamically and more efficiently, while also scaling well by allowing applications to process data in parallel, e.g., using a parallel computing platform **1030** (FIG. **10**). In at least one embodiment, rather than each application that shares a same functionality offered by a service **920** being required to have a respective instance of service **920**, service **920** may be shared between and among various applications. In at least one embodiment, services may include an inference server or engine that may be used for executing detection or segmentation tasks, as non-limiting examples. In at least one embodiment, a model training service may be included that may provide machine learning model training and/or retraining capabilities.

[0100] In at least one embodiment, where a service **920** includes an AI service (e.g., an inference service), one or more machine learning models associated with an application for anomaly detection (e.g., tumors, growth abnormalities, scarring, etc.) may be executed by calling upon (e.g., as an API call) an inference service (e.g., an inference server) to execute machine learning model(s), or processing thereof, as part of application execution. In at least one embodiment, where another application includes one or more machine learning models for segmentation tasks, an application may call upon an inference service to execute machine learning models for performing one or more of processing operations associated with segmentation tasks. In at least one embodiment, software **918** implementing advanced processing and inferencing pipeline may be streamlined because each application may call upon the same inference service to perform one or more inferencing tasks.

[0101] In at least one embodiment, hardware **922** may include GPUs, CPUs, graphics cards, an AI/deep learning system (e.g., an AI supercomputer, such as NVIDIA's DGX™ supercomputer system), a cloud platform, or a combination thereof. In at least one embodiment, different types of hardware **922** may be used to provide efficient, purpose-built support for software **918** and services **920** in deployment system **906**. In at least one embodiment, use of GPU processing may be implemented for processing locally (e.g., at facility **902**), within an AI/deep learning system, in a cloud system, and/or in other processing components of deployment system **906** to improve efficiency, accuracy, and efficacy of game name recognition.

[0102] In at least one embodiment, software **918** and/or services **920** may be optimized for GPU processing with respect to deep learning, machine learning, and/or high-performance computing, simulation, and visual computing, as non-limiting examples. In at least one embodiment, at least some of the computing environment of deployment system **906** and/or training system **904** may be executed in a datacenter or one or more supercomputers or high performance computing systems, with GPU-optimized software (e.g., hardware and software combination of NVIDIA's DGX™ system). In at least one embodiment, hardware **922** may include any number of GPUs that may be called upon to perform processing of data in parallel, as described herein. In at least one embodiment, cloud platform may further include GPU processing for GPU-optimized execution of deep learning tasks, machine learning tasks, or other computing tasks. In at least one embodiment, cloud platform (e.g., NVIDIA's NGC™) may be executed using an AI/deep learning supercomputer(s) and/or GPU-optimized software (e.g., as provided on NVIDIA's DGX™ systems) as a hardware abstraction and scaling platform. In at least one embodiment, cloud platform may integrate an application container clustering system or orchestration system (e.g., KUBERNETES) on multiple GPUs to enable seamless scaling and load balancing.

[0103] FIG. **10** is a system diagram for an example system **1000** for generating and deploying a deployment pipeline, according to at least one embodiment. In at least one embodiment, system **1000** may be used to implement process **900** of FIG. **9** and/or other processes including advanced processing and inferencing pipelines. In at least one embodiment, system **1000** may include training system **904** and deployment system **906**. In at least one embodiment, training system **904** and deployment system **906** may be implemented using software **918**, services **920**, and/or hardware **922**, as described herein.

[0104] In at least one embodiment, system **1000** (e.g., training system **904** and/or deployment system **906**) may implemented in a cloud computing environment (e.g., using cloud **1026**). In at least one embodiment, system **1000** may be implemented locally with respect to a facility, or as a combination of both cloud and local computing resources. In at least one embodiment, access to APIs in cloud **1026** may be restricted to authorized users through enacted security measures or protocols. In at least one embodiment, a security protocol may include web tokens that may be signed by an authentication (e.g., AuthN, AuthZ, Gluecon, etc.) service and may carry appropriate authorization. In at least one embodiment, APIs of virtual instruments (described herein), or other instantiations of system **1000**, may be restricted to a set of public internet service providers (ISPs) that have been vetted or authorized for interaction.

[0105] In at least one embodiment, various components of system **1000** may communicate between and among one another using any of a variety of different network types, including but not limited to local area networks (LANs) and/or wide area networks (WANs) via wired and/or wireless communication protocols. In at least one embodiment, communication between facilities and components of system **1000** (e.g., for transmitting inference requests, for receiving results of inference requests, etc.) may be communicated over a data bus or data busses, wireless data protocols (Wi-Fi), wired data protocols (e.g., Ethernet), etc.

[0106] In at least one embodiment, training system **904** may execute training pipelines **1004**, similar to those described herein with respect to FIG. **9**. In at least one embodiment, where one or more machine learning models are to be used in deployment pipelines **1010** by deployment system **906**, training pipelines **1004** may be used to train or retrain one or more (e.g., pre-trained) models, and/or implement one or more of pre-trained models **1006** (e.g., without a need for retraining or updating). In at least one embodiment, as a result of training pipelines **1004**, output model(s) **916** may be generated. In at least one embodiment, training pipelines **1004** may include any number of processing steps, AI-assisted annotation **910**, labeling or annotating of feedback data **908** to generate labeled data **912**, model selection from a model registry, model training **914**, training, retraining, or updating models, and/or other processing steps. In at least one embodiment, for different machine learning models used by deployment system **906**, different training pipelines **1004** may be used. In at least one embodiment, training pipeline **1004**, similar to a first example described with respect to FIG. **9**, may be used for a first machine learning model, training pipeline **1004**, similar to a second example described with respect to FIG. **9**, may be used for a second machine learning model, and training pipeline **1004**, similar to a third example described with respect to FIG. **9**, may be used for a third machine learning model. In at least one embodiment, any combination of tasks within training system **904** may be used depending on what is required for each respective machine learning model. In at least one embodiment, one or more of machine learning models may already be trained and ready for deployment so machine learning models may not undergo any processing by training system **904**, and may be implemented by deployment system **906**.

[0107] In at least one embodiment, output model(s) **916** and/or pre-trained model(s) **1006** may include any types of machine learning models depending on embodiment. In at least one embodiment, and without limitation, machine learning models used by system **1000** may include machine learning model(s) using linear regression, logistic regression, decision trees, support vector machines (SVM), Naïve Bayes, k-nearest neighbor (Knn), K means clustering, random forest, dimensionality reduction algorithms, gradient boosting algorithms, neural networks (e.g., auto-encoders, convolutional, recurrent, perceptrons, Long/Short Term Memory (LSTM), Bi-LSTM, Hopfield, Boltzmann, deep belief, deconvolutional, generative adversarial, liquid state machine, etc.), and/or other types of machine learning models.

[0108] In at least one embodiment, training pipelines **1004** may include AI-assisted annotation. In at least one embodiment, labeled data **912** (e.g., traditional annotation) may be generated by any number of techniques. In at least one embodiment, labels or other annotations may be generated within a drawing program (e.g., an annotation program), a computer aided design (CAD) program, a labeling program, another type of program suitable for generating annotations or labels for ground truth, and/or may be hand drawn, in some examples. In at least one embodiment, ground truth data may be synthetically produced (e.g., generated from computer models or renderings), real produced (e.g., designed and produced from real-world data), machine-automated (e.g., using feature analysis and learning to extract features from data and then generate labels), human annotated (e.g., labeler, or annotation expert, defines location of labels), and/or a combination thereof. In at least one embodiment, for each instance of feedback data **908** (or other data type used by machine learning models), there may be corresponding ground truth data generated by training system **904**. In at least one embodiment, AI-assisted annotation may be performed as part of deployment pipelines **1010**; either in addition to, or in lieu of, AI-assisted annotation included in training pipelines **1004**. In at least one embodiment, system **1000** may include a multi-layer platform that may include a software layer (e.g., software **918**) of diagnostic applications (or other application types) that may perform one or more medical imaging and diagnostic functions.

[0109] In at least one embodiment, a software layer may be implemented as a secure, encrypted, and/or authenticated API through which applications or containers may be invoked (e.g., called) from an external environment(s), e.g., facility **902**. In at least one embodiment, applications may then call or execute one or more services **920** for performing compute, AI, or visualization tasks associated with respective applications, and software **918** and/or services **920** may leverage hardware **922** to perform processing tasks in an effective and efficient manner.

[0110] In at least one embodiment, deployment system **906** may execute deployment pipelines **1010**. In at least one embodiment, deployment pipelines **1010** may include any number of applications that may be sequentially, non-sequentially, or otherwise applied to feedback data (and/or other data types), including AI-assisted annotation, as described above. In at least one embodiment, as described herein, a deployment pipeline **1010** for an individual device may be referred to as a virtual instrument for a device. In at least one embodiment, for a single device, there may be more than one deployment pipeline **1010** depending on information desired from data generated by a device.

[0111] In at least one embodiment, applications available for deployment pipelines **1010** may include any application that may be used for performing processing tasks on feedback data or other data from devices. In at least one embodiment, because various applications may share common image operations, in some embodiments, a data augmentation library (e.g., as one of services **920**) may be used to accelerate these operations. In at least one embodiment, to avoid bottlenecks of conventional processing approaches that rely on CPU processing, parallel computing platform **1030** may be used for GPU acceleration of these processing tasks.

[0112] In at least one embodiment, deployment system **906** may include a user interface (UI) **1014** (e.g., a graphical user interface, a web interface, etc.) that may be used to select applications for inclusion in deployment pipeline(s) **1010**, arrange applications, modify or change applications or parameters or constructs thereof, use and intera with deployment pipeline(s) **1010** during set-up and/or deployment, and/or to otherwise interact with deployment system **906**. In at least one embodiment, although not illustrated with respect to training system **904**, UI **1014** (or a different user interface) may be used for selecting models for use in deployment system **906**, for selecting models for training, or retraining, in training system **904**, and/or for otherwise interacting with training system **904**. In at least one embodiment, training system **904** and deployment system **906** may include DICOM adapters **1002**A and **1002**B.

[0113] In at least one embodiment, pipeline manager **1012** may be used, in addition to an application orchestration system **1028**, to manage interaction between applications or containers of deployment pipeline(s) **1010** and services **920** and/or hardware **922**. In at least one embodiment, pipeline manager **1012** may be configured to facilitate interactions from application to application, from application to service **920**, and/or from application or service to hardware **922**. In at least one embodiment, although illustrated as included in software **918**, this is not intended to be limiting, and in some examples pipeline manager **1012** may be included in services **920**. In at least one embodiment, application orchestration system **1028** (e.g., Kubernetes, DOCKER, etc.) may include a container orchestration system that may group applications into containers as logical units for coordination, management, scaling, and deployment. In at least one embodiment, by associating applications from deployment pipeline(s) **1010** (e.g., a reconstruction application, a segmentation application, etc.) with individual containers, each application may execute in a self-contained environment (e.g., at a kernel level) to increase speed and efficiency.

[0114] In at least one embodiment, each application and/or container (or image thereof) may be individually developed, modified, and deployed (e.g., a first user or developer may develop, modify, and deploy a first application and a second user or developer may develop, modify, and deploy a second application separate from a first user or developer), which may allow for focus on, and attention to, a task of a single application and/or container(s) without being hindered by tasks of other application(s) or container(s). In at least one embodiment, communication, and cooperation between different containers or applications may be aided by pipeline manager **1012** and application orchestration system **1028**. In at least one embodiment, so long as an expected input and/or output of each container or application is known by a system (e.g., based on constructs of applications or containers), application orchestration system **1028** and/or pipeline manager **1012** may facilitate communication among and between, and sharing of resources among and between, each of applications or containers. In at least one embodiment, because one or more of applications or containers in deployment pipeline(s) **1010** may share the same services and resources, application orchestration system **1028** may orchestrate, load balance, and determine sharing of services or resources between and among various applications or containers. In at least one embodiment, a scheduler may be used to track resource requirements of applications or containers, current usage or planned usage of these resources, and resource availability. In at least one embodiment, the scheduler may thus allocate resources to different applications and distribute resources between and among applications in view of requirements and availability of a system. In some examples, the scheduler (and/or other component of application orchestration system **1028**) may determine resource availability and distribution based on constraints imposed on a system (e.g., user constraints), such as quality of service (QoS), urgency of need for data outputs (e.g., to determine whether to execute real-time processing or delayed processing), etc.

[0115] In at least one embodiment, services **920** leveraged and shared by applications or containers in deployment system **906** may include compute services **1016**, collaborative content creation services **1017**, AI services **1018**, simulation services **1019**, visualization services **1020**, and/or other service types. In at least one embodiment, applications may call (e.g., execute) one or more of services **920** to perform processing operations for an application. In at least one embodiment, compute services **1016** may be leveraged by applications to perform super-computing or other high-performance computing (HPC) tasks. In at least one embodiment, compute service(s) **1016** may be leveraged to perform parallel processing (e.g., using a parallel computing platform **1030**) for processing data through one or more of applications and/or one or more tasks of a single application, substantially simultaneously. In at least one embodiment, parallel computing platform **1030** (e.g., NVIDIA's CUDA®) may enable general purpose computing on GPUs (GPGPU) (e.g., GPUs **1022**). In at least one embodiment, a software layer of parallel computing platform **1030** may provide access to virtual instruction sets and parallel computational elements of GPUs, for execution of compute kernels. In at least one embodiment, parallel computing platform **1030** may include memory and, in some embodiments, a memory may be shared between and among multiple containers, and/or between and among different processing tasks within a single container. In at least one embodiment, inter-process communication (IPC) calls may be generated for multiple containers and/or for multiple processes within a container to use same data from a shared segment of memory of parallel computing platform **1030** (e.g., where multiple different stages of an application or multiple applications are processing same information). In at least one embodiment, rather than making a copy of data and moving data to different locations in memory (e.g., a read/write operation), same data in the same location of a memory may be used for any number of processing tasks (e.g., at the same time, at different times, etc.). In at least one embodiment, as data is used to generate new data as a result of processing, this information of a new location of data may be stored and shared between various applications. In at least one embodiment, location of data and a location of updated or modified data may be part of a definition of how a payload is understood within containers.

[0116] In at least one embodiment, AI services **1018** may be leveraged to perform inferencing services for executing machine learning model(s) associated with applications (e.g., tasked with performing one or more processing tasks of an application). In at least one embodiment, AI services **1018** may leverage AI system **1024** to execute machine learning model(s) (e.g., neural networks, such as CNNs) for segmentation, reconstruction, object detection, feature detection, classification, and/or other inferencing tasks. In at least one embodiment, applications of deployment pipeline(s) **1010** may use one or more of output models **916** from training system **904** and/or other models of applications to perform inference on imaging data (e.g., DICOM data, RIS data, CIS data, REST compliant data, RPC data, raw data, etc.). In at least one embodiment, two or more examples of inferencing using application orchestration system **1028** (e.g., a scheduler) may be available. In at least one embodiment, a first category may include a high priority/low latency path that may achieve higher service level agreements, such as for performing inference on urgent requests during an emergency, or for a radiologist during diagnosis. In at least one embodiment, a second category may include a standard priority path that may be used for requests that may be non-urgent or where analysis may be performed at a later time. In at least one embodiment, application orches-

tration system **1028** may distribute resources (e.g., services **920** and/or hardware **922**) based on priority paths for different inferencing tasks of AI services **1018**.

[0117] In at least one embodiment, shared storage may be mounted to AI services **1018** within system **1000**. In at least one embodiment, shared storage may operate as a cache (or other storage device type) and may be used to process inference requests from applications. In at least one embodiment, when an inference request is submitted, a request may be received by a set of API instances of deployment system **906**, and one or more instances may be selected (e.g., for best fit, for load balancing, etc.) to process a request. In at least one embodiment, to process a request, a request may be entered into a database, a machine learning model may be located from model registry **924** if not already in a cache, a validation step may ensure appropriate machine learning model is loaded into a cache (e.g., shared storage), and/or a copy of a model may be saved to a cache. In at least one embodiment, the scheduler (e.g., of pipeline manager **1012**) may be used to launch an application that is referenced in a request if an application is not already running or if there are not enough instances of an application. In at least one embodiment, if an inference server is not already launched to execute a model, an inference server may be launched. In at least one embodiment, any number of inference servers may be launched per model. In at least one embodiment, in a pull model, in which inference servers are clustered, models may be cached whenever load balancing is advantageous. In at least one embodiment, inference servers may be statically loaded in corresponding, distributed servers.

[0118] In at least one embodiment, inferencing may be performed using an inference server that runs in a container. In at least one embodiment, an instance of an inference server may be associated with a model (and optionally a plurality of versions of a model). In at least one embodiment, if an instance of an inference server does not exist when a request to perform inference on a model is received, a new instance may be loaded. In at least one embodiment, when starting an inference server, a model may be passed to an inference server such that a same container may be used to serve different models so long as the inference server is running as a different instance.

[0119] In at least one embodiment, during application execution, an inference request for a given application may be received, and a container (e.g., hosting an instance of an inference server) may be loaded (if not already loaded), and a start procedure may be called. In at least one embodiment, pre-processing logic in a container may load, decode, and/or perform any additional pre-processing on incoming data (e.g., using a CPU(s) and/or GPU(s)). In at least one embodiment, once data is prepared for inference, a container may perform inference as necessary on data. In at least one embodiment, this may include a single inference call on one image (e.g., a hand X-ray), or may require inference on hundreds of images (e.g., a chest CT). In at least one embodiment, an application may summarize results before completing, which may include, without limitation, a single confidence score, pixel level-segmentation, voxel-level segmentation, generating a visualization, or generating text to summarize findings. In at least one embodiment, different models or applications may be assigned different priorities. For example, some models may have a real-time (turnaround time less than one minute) priority while others may have lower priority (e.g., turnaround less than 10 minutes). In at least one embodiment, model execution times may be measured from requesting institution or entity and may include partner network traversal time, as well as execution on an inference service.

[0120] In at least one embodiment, transfer of requests between services **920** and inference applications may be hidden behind a software development kit (SDK), and robust transport may be provided through a queue. In at least one embodiment, a request is placed in a queue via an API for an individual application/tenant ID combination and an SDK pulls a request from a queue and gives a request to an application. In at least one embodiment, a name of a queue may be provided in an environment from where an SDK picks up the request. In at least one embodiment, asynchronous communication through a queue may be useful as it may allow any instance of an application to pick up work as it becomes available. In at least one embodiment, results may be transferred back through a queue, to ensure no data is lost. In at least one embodiment, queues may also provide an ability to segment work, as highest priority work may go to a queue with most instances of an application connected to it, while lowest priority work may go to a queue with a single instance connected to it that processes tasks in an order received. In at least one embodiment, an application may run on a GPU-accelerated instance generated in cloud **1026**, and an inference service may perform inferencing on a GPU.

[0121] In at least one embodiment, visualization services **1020** may be leveraged to generate visualizations for viewing outputs of applications and/or deployment pipeline(s) **1010**. In at least one embodiment, GPUs **1022** may be leveraged by visualization services **1020** to generate visualizations. In at least one embodiment, rendering effects, such as ray-tracing or other light transport simulation techniques, may be implemented by visualization services **1020** to generate higher quality visualizations. In at least one embodiment, visualizations may include, without limitation, 2D image renderings, 3D volume renderings, 3D volume reconstruction, 2D tomographic slices, virtual reality displays, augmented reality displays, etc. In at least one embodiment, virtualized environments may be used to generate a virtual interactive display or environment (e.g., a virtual environment) for interaction by users of a system (e.g., doctors, nurses, radiologists, etc.). In at least one embodiment, visualization services **1020** may include an internal visualizer, cinematics, and/or other rendering or image processing capabilities or functionality (e.g., ray tracing, rasterization, internal optics, etc.).

[0122] In at least one embodiment, hardware **922** may include GPUs **1022**, AI system **1024**, cloud **1026**, and/or any other hardware used for executing training system **904** and/or deployment system **906**. In at least one embodiment, GPUs **1022** (e.g., NVIDIA's TESLA® and/or QUADRO® GPUs) may include any number of GPUs that may be used for executing processing tasks of compute services **1016**, collaborative content creation services **1017**, AI services **1018**, simulation services **1019**, visualization services **1020**, other services, and/or any of features or functionality of software **918**. For example, with respect to AI services **1018**, GPUs **1022** may be used to perform pre-processing on imaging data (or other data types used by machine learning models), post-processing on outputs of machine learning models, and/or to perform inferencing (e.g., to execute machine learning models). In at least one embodiment,

cloud **1026**, AI system **1024**, and/or other components of system **1000** may use GPUs **1022**. In at least one embodiment, cloud **1026** may include a GPU-optimized platform for deep learning tasks. In at least one embodiment, AI system **1024** may use GPUs, and cloud **1026**—or at least a portion tasked with deep learning or inferencing—may be executed using one or more AI systems **1024**. As such, although hardware **922** is illustrated as discrete components, this is not intended to be limiting, and any components of hardware **922** may be combined with, or leveraged by, any other components of hardware **922**.

[0123] In at least one embodiment, AI system **1024** may include a purpose-built computing system (e.g., a supercomputer or an HPC) configured for inferencing, deep learning, machine learning, and/or other artificial intelligence tasks. In at least one embodiment, AI system **1024** (e.g., NVIDIA's DGX™) may include GPU-optimized software (e.g., a software stack) that may be executed using a plurality of GPUs **1022**, in addition to CPUs, RAM, storage, and/or other components, features, or functionality. In at least one embodiment, one or more AI systems **1024** may be implemented in cloud **1026** (e.g., in a data center) for performing some or all of AI-based processing tasks of system **1000**.

[0124] In at least one embodiment, cloud **1026** may include a GPU-accelerated infrastructure (e.g., NVIDIA's NGC™) that may provide a GPU-optimized platform for executing processing tasks of system **1000**. In at least one embodiment, cloud **1026** may include an AI system(s) **1024** for performing one or more of AI-based tasks of system **1000** (e.g., as a hardware abstraction and scaling platform). In at least one embodiment, cloud **1026** may integrate with application orchestration system **1028** leveraging multiple GPUs to enable seamless scaling and load balancing between and among applications and services **920**. In at least one embodiment, cloud **1026** may be tasked with executing at least some of services **920** of system **1000**, including compute services **1016**, AI services **1018**, and/or visualization services **1020**, as described herein. In at least one embodiment, cloud **1026** may perform small and large batch inference (e.g., executing NVIDIA's TensorRT™), provide an accelerated parallel computing API and platform **1030** (e.g., NVIDIA's CUDA®), execute application orchestration system **1028** (e.g., KUBERNETES), provide a graphics rendering API and platform (e.g., for ray-tracing, 2D graphics, 3D graphics, and/or other rendering techniques to produce higher quality cinematics), and/or may provide other functionality for system **1000**.

[0125] In at least one embodiment, in an effort to preserve patient confidentiality (e.g., where patient data or records are to be used off-premises), cloud **1026** may include a registry, such as a deep learning container registry. In at least one embodiment, a registry may store containers for instantiations of applications that may perform pre-processing, post-processing, or other processing tasks on patient data. In at least one embodiment, cloud **1026** may receive data that includes patient data as well as sensor data in containers, perform requested processing for just sensor data in those containers, and then forward a resultant output and/or visualizations to appropriate parties and/or devices (e.g., on-premises medical devices used for visualization or diagnoses), all without having to extract, store, or otherwise access patient data. In at least one embodiment, confidentiality of patient data is preserved in compliance with HIPAA and/or other data regulations.

[0126] FIG. **11**A is a block diagram of an example generative language model system **1100** suitable for use in implementing at least some embodiments of the present disclosure. In the example illustrated in FIG. **11**A, the generative language model system **1100** includes a retrieval augmented generation (RAG) component **1192**, an input processor **1105**, a tokenizer **1110**, an embedding component **1120**, plug-ins/APIs **1195**, and a generative language model (LM) **1130** (which may include an LLM, a VLM, a multimodal LM, etc.).

[0127] At a high level, the input processor **1105** may receive an input **1101** comprising text and/or other types of input data (e.g., audio data, video data, image data, sensor data (e.g., LiDAR, RADAR, ultrasonic, etc.), 3D design data, CAD data, universal scene descriptor (USD) data, etc.), depending on the architecture of the generative LM **1130**. In some embodiments, the input **1101** includes plain text in the form of one or more sentences, paragraphs, and/or documents. Additionally or alternatively, the input **1101** may include numerical sequences, precomputed embeddings (e.g., word or sentence embeddings), and/or structured data (e.g., in tabular formats, JSON, or XML). In some embodiments in which the generative LM **1130** is capable of processing multimodal inputs, the input **1101** may combine text with image data, audio data, and/or other types of input data, such as but not limited to those described herein. Taking raw input text as an example, the input processor **1105** may prepare raw input text in various ways. For example, the input processor **1105** may perform various types of text cleaning to remove noise (e.g., special characters, punctuation, HTML tags, stopwords) from relevant textual content. In an example involving stopwords (common words that tend to carry little semantic meaning), the input processor **1105** may remove stopwords to reduce noise and focus the generative LM **1130** on more meaningful content. The input processor **1105** may apply text normalization, for example, by converting all characters to lowercase, removing accents, and/or or handling special cases like contractions or abbreviations to ensure consistency. These are just a few examples, and other types of input processing may be applied.

[0128] In some embodiments, a RAG component **1192** may be used to retrieve additional information to be used as part of the input **1101** or prompt. For example, in some embodiments, the input **1101** may be generated using the query or input to the model (e.g., a question, a request, etc.) in addition to data retrieved using the RAG component **1192**. In some embodiments, the input processor **1105** may analyze the input **1101** and communicate with the RAG component **1192** (or the RAG component **1192** may be part of the input processor **1105**, in embodiments) in order to identify relevant text and/or other data to provide to the generative LM **1130** as additional context or sources of information from which to identify the response, answer, or output **1190**, generally. For example, where the input indicates that the user is interested in a desired tire pressure for a particular make and model of vehicle, the RAG component **1192** may retrieve—using a vector search in an embedding space, for example—the tire pressure information or the text corresponding thereto from a digital (embedded) version of the user manual for that particular vehicle make and model.

Similarly, where a user revisits a chatbot related to a particular product offering or service, the RAG component **1192** may retrieve a prior stored conversation history—or at least a summary thereof—and include the prior conversation history along with the current ask/request as part of the input **1101** to the generative LM **1130**.

[0129] The tokenizer **1110** may segment the (e.g., processed) text into smaller units (tokens) for subsequent analysis and processing. The tokens may represent individual words, subwords, characters, etc., depending on the embodiment. Word-based tokenization divides the text into individual words, treating each word as a separate token. Subword tokenization breaks down words into smaller meaningful units (e.g., prefixes, suffixes, stems), enabling the generative LM **1130** to understand morphological variations and handle out-of-vocabulary words more effectively. Character-based tokenization represents each character as a separate token, enabling the generative LM **1130** to process text at a fine-grained level. The choice of tokenization strategy may depend on factors such as the language being processed, the task at hand, and/or characteristics of the training dataset. As such, the tokenizer **1110** may convert the (e.g., processed) text into a structured format according to tokenization schema being implemented in the particular embodiment.

[0130] The embedding component **1120** may use any known embedding technique to transform discrete tokens into (e.g., dense, continuous vector) representations of semantic meaning. For example, the embedding component **1120** may use pre-trained word embeddings (e.g., Word2Vec, GloVe, or FastText), one-hot encoding, Term Frequency-Inverse Document Frequency (TF-IDF) encoding, one or more embedding layers of a neural network, and/or otherwise.

[0131] In some embodiments in which the input **1101** includes image data, the input processor **1101** may resize the image data to a standard size compatible with format of a corresponding input channel and/or may normalize pixel values to a common range (e.g., 0 to 1) to ensure a consistent representation, and the embedding component **1120** may encode the image data using any known technique (e.g., using one or more convolutional neural networks (CNNs) to extract visual features). In some embodiments in which the input **1101** includes audio data, the input processor **1101** may resample an audio file to a consistent sampling rate for uniform processing, and the embedding component **1120** may use any known technique to extract and encode audio features-such as in the form of a spectrogram (e.g., a mel-spectrogram). In some embodiments in which the input **1101** includes video data, the input processor **1101** may extract frames or apply resizing to extracted frames, and the embedding component **1120** may extract features such as optical flow embeddings or video embeddings and/or may encode temporal information or sequences of frames. In some embodiments in which the input **1101** includes multimodal data, the embedding component **1120** may fuse representations of the different types of data (e.g., text, image, audio) using techniques like early fusion (concatenation), late fusion (sequential processing), attention-based fusion, etc.

[0132] The generative LM **1130** and/or other components of the generative LLM system **1100** may use different types of neural network architectures depending on the embodiment. For example, transformer-based architectures such as those used in models like GPT may be implemented, and may include self-attention mechanisms that weigh the importance of different words or tokens in the input sequence and/or feedforward networks that process the output of the self-attention layers, applying non-linear transformations to the input representations and extracting higher-level features. Some non-limiting example architectures include transformers (e.g., encoder-decoder, decoder only, multimodal), RNNs, LSTMs, fusion models, cross-modal embedding models that learn joint embedding spaces, graph neural networks (GNNs), hybrid architectures combining different types of architectures adversarial networks like generative adversarial networks or GANs or adversarial autoencoders (AAEs) for joint distribution learning, and others. As such, depending on the embodiment and architecture, the embedding component **1120** may apply an encoded representation of the input **1101** to the generative LM **1130**, and the generative LM **1130** may process the encoded representation of the input **1101** to generate an output **1190**, which may include responsive text and/or other types of data.

[0133] As described herein, in some embodiments, the generative LM **1130** may be configured to access or use—or capable of accessing or using—plug-ins/APIs **1195** (which may include one or more plug-ins, application programming interfaces (APIs), databases, data stores, repositories, etc.). For example, for certain tasks or operations that the generative LM **1130** is not ideally suited for, the model may have instructions (e.g., as a result of training, and/or based on instructions in a given prompt, such as those retrieved using the RAG component **1192**) to access one or more plug-ins/APIs **1195** (e.g., 3rd party plugins) for help in processing the current input. In such an example, where at least part of a prompt is related to restaurants or weather, the model may access one or more restaurant or weather plug-ins (e.g., via one or more APIs), send at least a portion of the prompt related to the particular plug-in/API **1195** to the plug-in/API **1195**, the plug-in/API **1195** may process the information and return an answer to the generative LM **1130**, and the generative LM **1130** may use the response to generate the output **1190**. This process may be repeated—e.g., recursively—for any number of iterations and using any number of plug-ins/APIs **1195** until an output **1190** that addresses each ask/question/request/process/operation/etc from the input **1101** can be generated. As such, the model(s) may not only rely on its own knowledge from training on a large dataset(s) and/or from data retrieved using the RAG component **1192**, but also on the expertise or optimized nature of one or more external resources-such as the plug-ins/APIs **1195**.

[0134] FIG. **11**B is a block diagram of an example embodiment in which the generative LM **1130** includes a transformer encoder-decoder, according to at least one embodiment. For example, assume input text such as "Who discovered gravity" is tokenized (e.g., by the tokenizer **1110** of FIG. **11**A) into tokens such as words, and each token is encoded (e.g., by the embedding component **1120** of FIG. **911**A) into a corresponding embedding (e.g., of size **512**). Since these token embeddings typically do not represent the position of the token in the input sequence, any known technique may be used to add a positional encoding to each token embedding to encode the sequential relationships and context of the tokens in the input sequence. As such, the

(e.g., resulting) embeddings may be applied to one or more encoder(s) **1135** of the generative LM **1130**.

[0135] In an example embodiment, the encoder(s) **1135** forms an encoder stack, where each encoder includes a self-attention layer and a feedforward network. In an example transformer architecture, each token (e.g., word) flows through a separate path. As such, each encoder may accept a sequence of vectors, passing each vector through the self-attention layer, then the feedforward network, and then upwards to the next encoder in the stack. Any known self-attention technique may be used. For example, to calculate a self-attention score for each token (word), a query vector, a key vector, and a value vector may be created for each token, a self-attention score may be calculated for pairs of tokens by taking the dot product of the query vector with the corresponding key vectors, normalizing the resulting scores, multiplying by corresponding value vectors, and summing weighted value vectors. The encoder may apply multi-headed attention in which the attention mechanism is applied multiple times in parallel with different learned weight matrices. Any number of encoders may be cascaded to generate a context vector encoding the input. An attention projection layer **1140** may convert the context vector into attention vectors (keys and values) for the decoder(s) **1145**.

[0136] In an example embodiment, the decoder(s) **1145** form a decoder stack, where each decoder includes a self-attention layer, an encoder-decoder self-attention layer that uses the attention vectors (keys and values) from the encoder to focus on relevant parts of the input sequence, and a feedforward network. As with the encoder(s) **1135**, in an example transformer architecture, each token (e.g., word) flows through a separate path in the decoder(s) **1145**. During a first pass, the decoder(s) **1145**, a classifier **1150**, and a generation mechanism **1155** may generate a first token, and the generation mechanism **1155** may apply the generated token as an input during a second pass. The process may repeat in a loop, successively generating and adding tokens (e.g., words) to the output from the preceding pass and applying the token embeddings of the composite sequence with positional encodings as an input to the decoder(s) **1145** during a subsequent pass, sequentially generating one token at a time (known as auto-regression) until predicting a symbol or token that represents the end of the response. Within each decoder, the self-attention layer is typically constrained to attend only to preceding positions in the output sequence by applying a masking technique (e.g., setting future positions to negative infinity) before the softmax operation. In an example embodiment, the encoder-decoder attention layer operates similarly to the (e.g., multi-headed) self-attention in the encoder(s) **1135**, except that it creates its queries from the layer below it and takes the keys and values (e.g., matrix) from the output of the encoder(s) **1135**.

[0137] As such, the decoder(s) **1145** may output some decoded (e.g., vector) representation of the input being applied during a particular pass. The classifier **1150** may include a multi-class classifier comprising one or more neural network layers that project the decoded (e.g., vector) representation into a corresponding dimensionality (e.g., one dimension for each supported word or token in the output vocabulary) and a softmax operation that converts logits to probabilities. As such, the generation mechanism **1155** may select or sample a word or token based on a corresponding predicted probability (e.g., select the word with the highest predicted probability) and append it to the output from a previous pass, generating each word or token sequentially. The generation mechanism **1155** may repeat the process, triggering successive decoder inputs and corresponding predictions until selecting or sampling a symbol or token that represents the end of the response, at which point, the generation mechanism **1155** may output the generated response.

[0138] FIG. **11**C is a block diagram of an example embodiment in which the generative LM **1130** includes a decoder-only transformer architecture, according to at least one embodiment. For example, the decoder(s) **1160** of FIG. **11**C may operate similarly as the decoder(s) **1145** of FIG. **11**B except each of the decoder(s) **1160** of FIG. **11**C omits the encoder-decoder self-attention layer (since there is no encoder in this embodiment). As such, the decoder(s) **1160** may form a decoder stack, where each decoder includes a self-attention layer and a feedforward network. Furthermore, instead of encoding the input sequence, a symbol or token representing the end of the input sequence (or the beginning of the output sequence) may be appended to the input sequence, and the resulting sequence (e.g., corresponding embeddings with positional encodings) may be applied to the decoder(s) **1160**. As with the decoder(s) **1145** of FIG. **11**B, each token (e.g., word) may flow through a separate path in the decoder(s) **1160**, and the decoder(s) **1160**, a classifier **1165**, and a generation mechanism **1170** may use auto-regression to sequentially generate one token at a time until predicting a symbol or token that represents the end of the response. The classifier **1165** and the generation mechanism **1170** may operate similarly as the classifier **1150** and the generation mechanism **1155** of FIG. **11**B, with the generation mechanism **1170** selecting or sampling each successive output token based on a corresponding predicted probability and appending it to the output from a previous pass, generating each token sequentially until selecting or sampling a symbol or token that represents the end of the response. These and other architectures described herein are meant simply as examples, and other suitable architectures may be implemented within the scope of the present disclosure.

[0139] FIG. **12** is a block diagram of an example computing device(s) **1200** suitable for use in implementing some embodiments of the present disclosure. Computing device **1200** may include an interconnect system **1202** that directly or indirectly couples the following devices: memory **1204**, one or more central processing units (CPUs) **1206**, one or more graphics processing units (GPUs) **1208**, a communication interface **1210**, input/output (I/O) ports **1212**, input/output components **1214**, a power supply **1216**, one or more presentation components **1218** (e.g., display(s)), and one or more logic units **1220**. In at least one embodiment, the computing device(s) **1200** may comprise one or more virtual machines (VMs), and/or any of the components thereof may comprise virtual components (e.g., virtual hardware components). For non-limiting examples, one or more of the GPUs **1208** may comprise one or more vGPUs, one or more of the CPUs **1206** may comprise one or more vCPUs, and/or one or more of the logic units **1220** may comprise one or more virtual logic units. As such, a computing device(s) **1200** may include discrete components (e.g., a full GPU dedicated to the computing device **1200**), virtual components (e.g., a portion of a GPU dedicated to the computing device **1200**), or a combination thereof.

[0140] Although the various blocks of FIG. **12** are shown as connected via the interconnect system **1202** with lines, this is not intended to be limiting and is for clarity only. For example, in some embodiments, a presentation component **1218**, such as a display device, may be considered an I/O component **1214** (e.g., if the display is a touch screen). As another example, the CPUs **1206** and/or GPUs **1208** may include memory (e.g., the memory **1204** may be representative of a storage device in addition to the memory of the GPUs **1208**, the CPUs **1206**, and/or other components). As such, the computing device of FIG. **12** is merely illustrative. Distinction is not made between such categories as "workstation," "server," "laptop," "desktop," "tablet," "client device," "mobile device," "hand-held device," "game console," "electronic control unit (ECU)," "virtual reality system," and/or other device or system types, as all are contemplated within the scope of the computing device of FIG. **12**.

[0141] The interconnect system **1202** may represent one or more links or busses, such as an address bus, a data bus, a control bus, or a combination thereof. The interconnect system **1202** may include one or more bus or link types, such as an industry standard architecture (ISA) bus, an extended industry standard architecture (EISA) bus, a video electronics standards association (VESA) bus, a peripheral component interconnect (PCI) bus, a peripheral component interconnect express (PCIe) bus, and/or another type of bus or link. In some embodiments, there are direct connections between components. As an example, the CPU **1206** may be directly connected to the memory **1204**. Further, the CPU **1206** may be directly connected to the GPU **1208**. Where there is direct, or point-to-point connection between components, the interconnect system **1202** may include a PCIe link to carry out the connection. In these examples, a PCI bus need not be included in the computing device **1200**.

[0142] The memory **1204** may include any of a variety of computer-readable media. The computer-readable media may be any available media that may be accessed by the computing device **1200**. The computer-readable media may include both volatile and nonvolatile media, and removable and non-removable media. By way of example, and not limitation, the computer-readable media may comprise computer-storage media and communication media.

[0143] The computer-storage media may include both volatile and nonvolatile media and/or removable and non-removable media implemented in any method or technology for storage of information such as computer-readable instructions, data structures, program modules, and/or other data types. For example, the memory **1204** may store computer-readable instructions (e.g., that represent a program(s) and/or a program element(s), such as an operating system. Computer-storage media may include, but is not limited to, RAM, ROM, EEPROM, flash memory or other memory technology, CD-ROM, digital versatile disks (DVD) or other optical disk storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or any other medium which may be used to store the desired information and which may be accessed by computing device **1200**. As used herein, computer storage media does not comprise signals per sc.

[0144] The computer storage media may embody computer-readable instructions, data structures, program modules, and/or other data types in a modulated data signal such as a carrier wave or other transport mechanism and includes any information delivery media. The term "modulated data signal" may refer to a signal that has one or more of its characteristics set or changed in such a manner as to encode information in the signal. By way of example, and not limitation, the computer storage media may include wired media such as a wired network or direct-wired connection, and wireless media such as acoustic, RF, infrared and other wireless media. Combinations of any of the above should also be included within the scope of computer-readable media.

[0145] The CPU(s) **1206** may be configured to execute at least some of the computer-readable instructions to control one or more components of the computing device **1200** to perform one or more of the methods and/or processes described herein. The CPU(s) **1206** may each include one or more cores (e.g., one, two, four, eight, twenty-eight, seventy-two, etc.) that are capable of handling a multitude of software threads simultaneously. The CPU(s) **1206** may include any type of processor, and may include different types of processors depending on the type of computing device **1200** implemented (e.g., processors with fewer cores for mobile devices and processors with more cores for servers). For example, depending on the type of computing device **1200**, the processor may be an Advanced RISC Machines (ARM) processor implemented using Reduced Instruction Set Computing (RISC) or an x86 processor implemented using Complex Instruction Set Computing (CISC). The computing device **1200** may include one or more CPUs **1206** in addition to one or more microprocessors or supplementary co-processors, such as math co-processors.

[0146] In addition to or alternatively from the CPU(s) **1206**, the GPU(s) **1208** may be configured to execute at least some of the computer-readable instructions to control one or more components of the computing device **1200** to perform one or more of the methods and/or processes described herein. One or more of the GPU(s) **1208** may be an integrated GPU (e.g., with one or more of the CPU(s) **1206** and/or one or more of the GPU(s) **1208** may be a discrete GPU. In embodiments, one or more of the GPU(s) **1208** may be a coprocessor of one or more of the CPU(s) **1206**. The GPU(s) **1208** may be used by the computing device **1200** to render graphics (e.g., 3D graphics) or perform general purpose computations. For example, the GPU(s) **1208** may be used for General-Purpose computing on GPUs (GPGPU). The GPU(s) **1208** may include hundreds or thousands of cores that are capable of handling hundreds or thousands of software threads simultaneously. The GPU(s) **1208** may generate pixel data for output images in response to rendering commands (e.g., rendering commands from the CPU(s) **1206** received via a host interface). The GPU(s) **1208** may include graphics memory, such as display memory, for storing pixel data or any other suitable data, such as GPGPU data. The display memory may be included as part of the memory **1204**. The GPU(s) **1208** may include two or more GPUs operating in parallel (e.g., via a link). The link may directly connect the GPUs (e.g., using NVLINK) or may connect the GPUs through a switch (e.g., using NVSwitch). When combined together, each GPU **1208** may generate pixel data or GPGPU data for different portions of an output or for different outputs (e.g., a first GPU for a first image and a second GPU for a second image). Each GPU may include its own memory, or may share memory with other GPUs.

[0147] In addition to or alternatively from the CPU(s) **1206** and/or the GPU(s) **1208**, the logic unit(s) **1220** may be configured to execute at least some of the computer-readable instructions to control one or more components of the computing device **1200** to perform one or more of the methods and/or processes described herein. In embodiments, the CPU(s) **1206**, the GPU(s) **1208**, and/or the logic unit(s) **1220** may discretely or jointly perform any combination of the methods, processes and/or portions thereof. One or more of the logic units **1220** may be part of and/or integrated in one or more of the CPU(s) **1206** and/or the GPU(s) **1208** and/or one or more of the logic units **1220** may be discrete components or otherwise external to the CPU(s) **1206** and/or the GPU(s) **1208**. In embodiments, one or more of the logic units **1220** may be a coprocessor of one or more of the CPU(s) **1206** and/or one or more of the GPU(s) **1208**.

[0148] Examples of the logic unit(s) **1220** include one or more processing cores and/or components thereof, such as Data Processing Units (DPUs), Tensor Cores (TCs), Tensor Processing Units (TPUs), Pixel Visual Cores (PVCs), Vision Processing Units (VPUs), Graphics Processing Clusters (GPCs), Texture Processing Clusters (TPCs), Streaming Multiprocessors (SMs), Tree Traversal Units (TTUs), Artificial Intelligence Accelerators (AIAs), Deep Learning Accelerators (DLAs), Programmable Vision Accelerator (PVAs)—which may include one or more direct memory access (DMA) systems, one or more vision or vector processing units (VPUs), one or more pixel processing engines (PPEs), one or more decoupled accelerators (e.g., decoupled lookup table (DLUT) accelerators), etc., Vision Processing Units (VPUs), Optical Flow Accelerators (OFAs), Field Programmable Gate Arrays (FPGAs), Neuromorphic Chips, Quantum Processing Units (QPUs), Associative Process Units (APUs), Arithmetic-Logic Units (ALUs), Application-Specific Integrated Circuits (ASICs), Floating Point Units (FPUs), input/output (I/O) elements, peripheral component interconnect (PCI) or peripheral component interconnect express (PCIe) elements, and/or the like.

[0149] The communication interface **1210** may include one or more receivers, transmitters, and/or transceivers that allow the computing device **1200** to communicate with other computing devices via an electronic communication network, included wired and/or wireless communications. The communication interface **1210** may include components and functionality to allow communication over any of a number of different networks, such as wireless networks (e.g., Wi-Fi, Z-Wave, Bluetooth, Bluetooth LE, ZigBee, etc.), wired networks (e.g., communicating over Ethernet or InfiniBand), low-power wide-area networks (e.g., LoRaWAN, SigFox, etc.), and/or the Internet. In one or more embodiments, logic unit(s) **1220** and/or communication interface **1210** may include one or more data processing units (DPUs) to transmit data received over a network and/or through interconnect system **1202** directly to (e.g., a memory of) one or more GPU(s) **1208**.

[0150] The I/O ports **1212** may allow the computing device **1200** to be logically coupled to other devices including the I/O components **1214**, the presentation component(s) **1218**, and/or other components, some of which may be built in to (e.g., integrated in) the computing device **1200**. Illustrative I/O components **1214** include a microphone, mouse, keyboard, joystick, game pad, game controller, satellite dish, scanner, printer, wireless device, etc. The I/O components **1214** may provide a natural user interface (NUI) that processes air gestures, voice, or other physiological inputs generated by a user. In some instances, inputs may be transmitted to an appropriate network element for further processing. An NUI may implement any combination of speech recognition, stylus recognition, facial recognition, biometric recognition, gesture recognition both on screen and adjacent to the screen, air gestures, head and eye tracking, and touch recognition (as described in more detail below) associated with a display of the computing device **1200**. The computing device **1200** may be include depth cameras, such as stereoscopic camera systems, infrared camera systems, RGB camera systems, touchscreen technology, and combinations of these, for gesture detection and recognition. Additionally, the computing device **1200** may include accelerometers or gyroscopes (e.g., as part of an inertia measurement unit (IMU)) that allow detection of motion. In some examples, the output of the accelerometers or gyroscopes may be used by the computing device **1200** to render immersive augmented reality or virtual reality.

[0151] The power supply **1216** may include a hard-wired power supply, a battery power supply, or a combination thereof. The power supply **1216** may provide power to the computing device **1200** to allow the components of the computing device **1200** to operate.

[0152] The presentation component(s) **1218** may include a display (e.g., a monitor, a touch screen, a television screen, a heads-up-display (HUD), other display types, or a combination thereof), speakers, and/or other presentation components. The presentation component(s) **1218** may receive data from other components (e.g., the GPU(s) **1208**, the CPU(s) **1206**, DPUs, etc.), and output the data (e.g., as an image, video, sound, etc.).

Example Data Center

[0153] FIG. **13** illustrates an example data center **1300** that may be used in at least one embodiments of the present disclosure. The data center **1300** may include a data center infrastructure layer **1310**, a framework layer **1320**, a software layer **1330**, and/or an application layer **1340**.

[0154] As shown in FIG. **13**, the data center infrastructure layer **1310** may include a resource orchestrator **1312**, grouped computing resources **1314**, and node computing resources ("node C.R.s") **1316(1)-1316**(N), where "N" represents any whole, positive integer. In at least one embodiment, node C.R.s **1316(1)-1316**(N) may include, but are not limited to, any number of central processing units (CPUs) or other processors (including DPUs, accelerators, field programmable gate arrays (FPGAs), graphics processors or graphics processing units (GPUs), etc.), memory devices (e.g., dynamic read-only memory), storage devices (e.g., solid state or disk drives), network input/output (NW I/O) devices, network switches, virtual machines (VMs), power modules, and/or cooling modules, etc. In some embodiments, one or more node C.R.s from among node C.R.s **1316(1)-1316**(N) may correspond to a server having one or more of the above-mentioned computing resources. In addition, in some embodiments, the node C.R.s **1316(1)-13161** (N) may include one or more virtual components, such as vGPUs, vCPUs, and/or the like, and/or one or more of the node C.R.s **1316(1)-1316**(N) may correspond to a virtual machine (VM).

[0155] In at least one embodiment, grouped computing resources **1314** may include separate groupings of node C.R.s **1316** housed within one or more racks (not shown), or

many racks housed in data centers at various geographical locations (also not shown). Separate groupings of node C.R.s **1316** within grouped computing resources **1314** may include grouped compute, network, memory or storage resources that may be configured or allocated to support one or more workloads. In at least one embodiment, several node C.R.s **1316** including CPUs, GPUs, DPUs, and/or other processors may be grouped within one or more racks to provide compute resources to support one or more workloads. The one or more racks may also include any number of power modules, cooling modules, and/or network switches, in any combination.

[0156] The resource orchestrator **1312** may configure or otherwise control one or more node C.R.s **1316**(**1**)-**1316**(N) and/or grouped computing resources **1314**. In at least one embodiment, resource orchestrator **1312** may include a software design infrastructure (SDI) management entity for the data center **1300**. The resource orchestrator **1312** may include hardware, software, or some combination thereof.

[0157] In at least one embodiment, as shown in FIG. **13**, framework layer **1320** may include a job scheduler **1328**, a configuration manager **1334**, a resource manager **1336**, and/or a distributed file system **1338**. The framework layer **1320** may include a framework to support software **1332** of software layer **1330** and/or one or more application(s) **1342** of application layer **1340**. The software **1332** or application (s) **1342** may respectively include web-based service software or applications, such as those provided by Amazon Web Services, Google Cloud and Microsoft Azure. The framework layer **1320** may be, but is not limited to, a type of free and open-source software web application framework such as Apache Spark™ (hereinafter "Spark") that may use distributed file system **1338** for large-scale data processing (e.g., "big data"). In at least one embodiment, job scheduler **1328** may include a Spark driver to facilitate scheduling of workloads supported by various layers of data center **1300**. The configuration manager **1334** may be capable of configuring different layers such as software layer **1330** and framework layer **1320** including Spark and distributed file system **1338** for supporting large-scale data processing. The resource manager **1336** may be capable of managing clustered or grouped computing resources mapped to or allocated for support of distributed file system **1338** and job scheduler **1328**. In at least one embodiment, clustered or grouped computing resources may include grouped computing resource **1314** at data center infrastructure layer **1310**. The resource manager **1336** may coordinate with resource orchestrator **1312** to manage these mapped or allocated computing resources.

[0158] In at least one embodiment, software **1332** included in software layer **1330** may include software used by at least portions of node C.R.s **1316**(**1**)-**1316**(N), grouped computing resources **1314**, and/or distributed file system **1338** of framework layer **1320**. One or more types of software may include, but are not limited to, Internet web page search software, e-mail virus scan software, database software, and streaming video content software.

[0159] In at least one embodiment, application(s) **1342** included in application layer **1340** may include one or more types of applications used by at least portions of node C.R.s **1316**(**1**)-**1316**(N), grouped computing resources **1314**, and/or distributed file system **1338** of framework layer **1320**. One or more types of applications may include, but are not limited to, any number of a genomics application, a cognitive compute, and a machine learning application, including training or inferencing software, machine learning framework software (e.g., PyTorch, TensorFlow, Caffe, etc.), and/or other machine learning applications used in conjunction with one or more embodiments.

[0160] In at least one embodiment, any of configuration manager **1334**, resource manager **1336**, and resource orchestrator **1312** may implement any number and type of self-modifying actions based on any amount and type of data acquired in any technically feasible fashion. Self-modifying actions may relieve a data center operator of data center **1300** from making possibly bad configuration decisions and possibly avoiding underutilized and/or poor performing portions of a data center.

[0161] The data center **1300** may include tools, services, software or other resources to train one or more machine learning models or predict or infer information using one or more machine learning models according to one or more embodiments described herein. For example, a machine learning model(s) may be trained by calculating weight parameters according to a neural network architecture using software and/or computing resources described above with respect to the data center **1300**. In at least one embodiment, trained or deployed machine learning models corresponding to one or more neural networks may be used to infer or predict information using resources described above with respect to the data center **1300** by using weight parameters calculated through one or more training techniques, such as but not limited to those described herein.

[0162] In at least one embodiment, the data center **1300** may use CPUs, application-specific integrated circuits (ASICs), GPUs, FPGAs, and/or other hardware (or virtual compute resources corresponding thereto) to perform training and/or inferencing using above-described resources. Moreover, one or more software and/or hardware resources described above may be configured as a service to allow users to train or performing inferencing of information, such as image recognition, speech recognition, or other artificial intelligence services.

Example Network Environments

[0163] Network environments suitable for use in implementing embodiments of the disclosure may include one or more client devices, servers, network attached storage (NAS), other backend devices, and/or other device types. The client devices, servers, and/or other device types (e.g., each device) may be implemented on one or more instances of the computing device(s) **1200** of FIG. **12**—e.g., each device may include similar components, features, and/or functionality of the computing device(s) **1200**. In addition, where backend devices (e.g., servers, NAS, etc.) are implemented, the backend devices may be included as part of a data center **1300**, an example of which is described in more detail herein with respect to FIG. **13**.

[0164] Components of a network environment may communicate with each other via a network(s), which may be wired, wireless, or both. The network may include multiple networks, or a network of networks. By way of example, the network may include one or more Wide Area Networks (WANs), one or more Local Area Networks (LANs), one or more public networks such as the Internet and/or a public switched telephone network (PSTN), and/or one or more private networks. Where the network includes a wireless telecommunications network, components such as a base

station, a communications tower, or even access points (as well as other components) may provide wireless connectivity.

[0165] Compatible network environments may include one or more peer-to-peer network environments—in which case a server may not be included in a network environment—and one or more client-server network environments—in which case one or more servers may be included in a network environment. In peer-to-peer network environments, functionality described herein with respect to a server(s) may be implemented on any number of client devices.

[0166] In at least one embodiment, a network environment may include one or more cloud-based network environments, a distributed computing environment, a combination thereof, etc. A cloud-based network environment may include a framework layer, a job scheduler, a resource manager, and a distributed file system implemented on one or more of servers, which may include one or more core network servers and/or edge servers. A framework layer may include a framework to support software of a software layer and/or one or more application(s) of an application layer. The software or application(s) may respectively include web-based service software or applications. In embodiments, one or more of the client devices may use the web-based service software or applications (e.g., by accessing the service software and/or applications via one or more application programming interfaces (APIs)). The framework layer may be, but is not limited to, a type of free and open-source software web application framework such as that may use a distributed file system for large-scale data processing (e.g., "big data").

[0167] A cloud-based network environment may provide cloud computing and/or cloud storage that carries out any combination of computing and/or data storage functions described herein (or one or more portions thereof). Any of these various functions may be distributed over multiple locations from central or core servers (e.g., of one or more data centers that may be distributed across a state, a region, a country, the globe, etc.). If a connection to a user (e.g., a client device) is relatively close to an edge server(s), a core server(s) may designate at least a portion of the functionality to the edge server(s). A cloud-based network environment may be private (e.g., limited to a single organization), may be public (e.g., available to many organizations), and/or a combination thereof (e.g., a hybrid cloud environment).

[0168] The client device(s) may include at least some of the components, features, and functionality of the example computing device(s) **1200** described herein with respect to FIG. **12**. By way of example and not limitation, a client device may be embodied as a Personal Computer (PC), a laptop computer, a mobile device, a smartphone, a tablet computer, a smart watch, a wearable computer, a Personal Digital Assistant (PDA), an MP3 player, a virtual reality headset, a Global Positioning System (GPS) or device, a video player, a video camera, a surveillance device or system, a vehicle, a boat, a flying vessel, a virtual machine, a drone, a robot, a handheld communications device, a hospital device, a gaming device or system, an entertainment system, a vehicle computer system, an embedded system controller, a remote control, an appliance, a consumer electronic device, a workstation, an edge device, any combination of these delineated devices, or any other suitable device.

[0169] The disclosure may be described in the general context of computer code or machine-useable instructions, including computer-executable instructions such as program modules, being executed by a computer or other machine, such as a personal data assistant or other handheld device. Generally, program modules including routines, programs, objects, components, data structures, etc., refer to code that perform particular tasks or implement particular abstract data types. The disclosure may be practiced in a variety of system configurations, including hand-held devices, consumer electronics, general-purpose computers, more specialty computing devices, etc. The disclosure may also be practiced in distributed computing environments where tasks are performed by remote-processing devices that are linked through a communications network.

[0170] Other variations are within the spirit of present disclosure. Thus, while disclosed techniques are susceptible to various modifications and alternative constructions, certain illustrated embodiments thereof are shown in drawings and have been described above in detail. It should be understood, however, that there is no intention to limit disclosure to specific form or forms disclosed, but on contrary, intention is to cover all modifications, alternative constructions, and equivalents falling within spirit and scope of disclosure, as defined in appended claims.

[0171] Use of terms "a" and "an" and "the" and similar referents in context of describing disclosed embodiments (especially in context of following claims) are to be construed to cover both singular and plural, unless otherwise indicated herein or clearly contradicted by context, and not as a definition of a term. Terms "comprising," "having," "including," and "containing" are to be construed as open-ended terms (meaning "including, but not limited to,") unless otherwise noted. "Connected," when unmodified and referring to physical connections, is to be construed as partly or wholly contained within, attached to, or joined together, even if there is something intervening. Recitation of ranges of values herein are merely intended to serve as a shorthand method of referring individually to each separate value falling within range, unless otherwise indicated herein and each separate value is incorporated into specification as if it were individually recited herein. In at least one embodiment, use of the term "set" (e.g., "a set of items") or "subset" unless otherwise noted or contradicted by context, is to be construed as a nonempty collection comprising one or more members. Further, unless otherwise noted or contradicted by context, the term "subset" of a corresponding set does not necessarily denote a proper subset of the corresponding set, but subset and corresponding set may be equal.

[0172] Conjunctive language, such as phrases of form "at least one of A, B, and C," or "at least one of A, B and C," unless specifically stated otherwise or otherwise clearly contradicted by context, is otherwise understood with context as used in general to present that an item, term, etc., may be either A or B or C, or any nonempty subset of set of A and B and C. For instance, in illustrative example of a set having three members, conjunctive phrases "at least one of A, B, and C" and "at least one of A, B and C" refer to any of following sets: {A}, {B}, {C}, {A, B}, {A, C}, {B, C}, {A, B, C}. Thus, such conjunctive language is not generally intended to imply that certain embodiments require at least one of A, at least one of B and at least one of C each to be present. In addition, unless otherwise noted or contradicted by context, the term "plurality" indicates a state of being

plural (e.g., "a plurality of items" indicates multiple items). In at least one embodiment, a number of items in a plurality is at least two, but can be more when so indicated either explicitly or by context. Further, unless stated otherwise or otherwise clear from context, the phrase "based on" means "based at least in part on" and not "based solely on."

[0173] Operations of processes described herein can be performed in any suitable order unless otherwise indicated herein or otherwise clearly contradicted by context. In at least one embodiment, a process such as those processes described herein (or variations and/or combinations thereof) is performed under control of one or more computer systems configured with executable instructions and is implemented as code (e.g., executable instructions, one or more computer programs or one or more applications) executing collectively on one or more processors, by hardware or combinations thereof. In at least one embodiment, code is stored on a computer-readable storage medium, for example, in the form of a computer program comprising a plurality of instructions executable by one or more processors. In at least one embodiment, a computer-readable storage medium is a non-transitory computer-readable storage medium that excludes transitory signals (e.g., a propagating transient electric or electromagnetic transmission) but includes non-transitory data storage circuitry (e.g., buffers, cache, and queues) within transceivers of transitory signals. In at least one embodiment, code (e.g., executable code or source code) is stored on a set of one or more non-transitory computer-readable storage media having stored thereon executable instructions (or other memory to store executable instructions) that, when executed (i.e., as a result of being executed) by one or more processors of a computer system, cause computer system to perform operations described herein. In at least one embodiment, set of non-transitory computer-readable storage media comprises multiple non-transitory computer-readable storage media and one or more of individual non-transitory storage media of multiple non-transitory computer-readable storage media lack all of code while multiple non-transitory computer-readable storage media collectively store all of code. In at least one embodiment, executable instructions are executed such that different instructions are executed by different processors—for example, a non-transitory computer-readable storage medium store instructions and a main central processing unit ("CPU") executes some of instructions while a graphics processing unit ("GPU") executes other instructions. In at least one embodiment, different components of a computer system have separate processors and different processors execute different subsets of instructions.

[0174] Accordingly, in at least one embodiment, computer systems are configured to implement one or more services that singly or collectively perform operations of processes described herein and such computer systems are configured with applicable hardware and/or software that enable performance of operations. Further, a computer system that implements at least one embodiment of present disclosure is a single device and, in another embodiment, is a distributed computer system comprising multiple devices that operate differently such that distributed computer system performs operations described herein and such that a single device does not perform all operations.

[0175] Use of any and all examples, or exemplary language (e.g., "such as") provided herein, is intended merely to better illuminate embodiments of disclosure and does not pose a limitation on scope of disclosure unless otherwise claimed. No language in specification should be construed as indicating any non-claimed element as essential to practice of disclosure.

[0176] All references, including publications, patent applications, and patents, cited herein are hereby incorporated by reference to the same extent as if each reference were individually and specifically indicated to be incorporated by reference and were set forth in its entirety herein.

[0177] In description and claims, terms "coupled" and "connected," along with their derivatives, may be used. It should be understood that these terms may be not intended as synonyms for each other. Rather, in particular examples, "connected" or "coupled" may be used to indicate that two or more elements are in direct or indirect physical or electrical contact with each other. "Coupled" may also mean that two or more elements are not in direct contact with each other, but yet still co-operate or interact with each other.

[0178] Unless specifically stated otherwise, it may be appreciated that throughout specification terms such as "processing," "computing," "calculating," "determining," or like, refer to action and/or processes of a computer or computing system, or similar electronic computing device, that manipulate and/or transform data represented as physical, such as electronic, quantities within computing system's registers and/or memories into other data similarly represented as physical quantities within computing system's memories, registers or other such information storage, transmission or display devices.

[0179] In a similar manner, the term "processor" may refer to any device or portion of a device that processes electronic data from registers and/or memory and transforms that electronic data into other electronic data that may be stored in registers and/or memory. As non-limiting examples, "processor" may be a CPU or a GPU. A "computing platform" may comprise one or more processors. As used herein, "software" processes may include, for example, software and/or hardware entities that perform work over time, such as tasks, threads, and intelligent agents. Also, each process may refer to multiple processes, for carrying out instructions in sequence or in parallel, continuously or intermittently. In at least one embodiment, terms "system" and "method" are used herein interchangeably insofar as a system may embody one or more methods and methods may be considered a system.

[0180] In the present document, references may be made to obtaining, acquiring, receiving, or inputting analog or digital data into a subsystem, computer system, or computer-implemented machine. In at least one embodiment, a process of obtaining, acquiring, receiving, or inputting analog and digital data can be accomplished in a variety of ways such as by receiving data as a parameter of a function call or a call to an application programming interface. In at least one embodiment, processes of obtaining, acquiring, receiving, or inputting analog or digital data can be accomplished by transferring data via a serial or parallel interface. In at least one embodiment, processes of obtaining, acquiring, receiving, or inputting analog or digital data can be accomplished by transferring data via a computer network from providing entity to acquiring entity. In at least one embodiment, references may also be made to providing, outputting, transmitting, sending, or presenting analog or digital data. In various examples, processes of providing, outputting, transmitting, sending, or presenting analog or digital data can be

accomplished by transferring data as an input or output parameter of a function call, a parameter of an application programming interface or interprocess communication mechanism.

[0181] Although descriptions herein set forth example embodiments of described techniques, other architectures may be used to implement described functionality, and are intended to be within scope of this disclosure. Furthermore, although specific distributions of responsibilities may be defined above for purposes of description, various functions and responsibilities might be distributed and divided in different ways, depending on circumstances.

[0182] Furthermore, although subject matter has been described in language specific to structural features and/or methodological acts, it is to be understood that subject matter claimed in appended claims is not necessarily limited to specific features or acts described. Rather, specific features and acts are disclosed as exemplary forms of implementing the claims.

What is claimed is:

1. A method comprising:

performing a plurality of iterations of an outer processing loop to identify a plurality of content units (CUs) of a media item having a plurality of frames, an individual iteration of the plurality of iterations of the outer processing loop comprising:

updating, using a first neural network (NN) and an identified non-blank CU, a state of the media item; and

performing one or more iterations of an inner processing loop, an individual iteration of the one or more iterations of the inner processing loop comprising:

processing, using a second NN, the state of the media item and an individual frame of the plurality of frames to predict a CU of one or more CUs associated with the individual frame,

wherein the one or more iterations of the inner processing loop are performed until the predicted CU corresponds to a non-blank CU; and

generating, using the identified plurality of CUs, a representation of the media item.

2. The method of claim **1**, wherein the updating the state of the media item comprises:

processing, using the first NN, the state of the media item and the identified non-blank CU.

3. The method of claim **2**, wherein the first NN comprises a predictor NN.

4. The method of claim **1**, wherein the second NN comprises a joiner NN, and wherein the processing the state of the media item and the individual frame comprises:

processing, using the joiner NN, the state of the media item and an encoder vector to generate a prediction vector, wherein the encoder vector is generated by applying an encoder NN to the individual frame.

5. The method of claim **4**, wherein the processing the state of the media item and the individual frame further comprises:

generating, using a classifier NN and the prediction vector, a plurality of probabilities that the individual frame is associated with at least one of:

a plurality of vocabulary CUs, or

a blank CU.

6. The method of claim **1**, wherein the individual iteration of the one or more iterations of the inner processing loop further comprises:

maintaining, responsive to determining that the predicted CU corresponds to a blank CU, the state of the media item.

7. The method of claim **1**, wherein a next iteration of the plurality of iterations of the outer processing loop is initiated responsive to identification of the non-blank CU.

8. The method of claim **1**, wherein the media item comprises a speech utterance, and wherein the representation of the media item comprises a transcription of the speech utterance.

9. The method of claim **1**, further comprising:

setting, prior to a first iteration of the plurality of iterations of the outer processing loop, the identified non-blank CU to a default beginning-of-sequence (BOS) CU.

10. The method of claim **1**, wherein the plurality of iterations of the outer processing loop to identify the plurality of CUs of the media item is performed in parallel to a second plurality of iterations of the outer processing loop performed to identify a second plurality of CUs of a second media item, and wherein the plurality of iterations and the second plurality of iterations comprise an equal number of calls to the first NN to identify an equal number of CUs.

11. A system comprising:

one or more processors to:

perform a plurality of iterations of an outer processing loop to identify a plurality of content units (CUs) of a media item having a plurality of frames, wherein to perform an individual iteration of the plurality of iterations of the outer processing loop, the one or more processors are to:

update, using a first neural network (NN) and an identified non-blank CU, a state of the media item; and

perform one or more iterations of an inner processing loop, wherein to perform an individual iteration of the one or more iterations of the inner processing loop, the one or more processors are to:

process, using a second NN, the state of the media item and an individual frame of the plurality of frames to predict a CU of one or more CUs associated with the individual frame, wherein the one or more iterations of the inner processing loop are performed until the predicted CU corresponds to a non-blank CU; and

generate, using the identified plurality of CUs, a representation of the media item.

12. The system of claim **11**, wherein to update the state of the media item, the one or more processors are to:

process, using the first NN, the state of the media item and the identified non-blank CU.

13. The system of claim **11**, wherein the second NN comprises a joiner NN, and wherein to process the state of the media item and the individual frame, the one or more processors are to:

process, using the joiner NN, the state of the media item and an encoder vector to generate a prediction vector, wherein the encoder vector is generated by applying an encoder NN to the individual frame.

14. The system of claim **13**, wherein to process the state of the media item and the individual frame, the one or more processors are further to:

generate, using a classifier NN and the prediction vector, a plurality of probabilities that the individual frame is associated with at least one of:
a plurality of vocabulary CUs, or
a blank CU.

15. The system of claim **11**, wherein to perform the individual iteration of the one or more iterations of the inner processing loop, the one or more processors are further to:
maintain, responsive to determining that the predicted CU corresponds to a blank CU, the state of the media item.

16. The system of claim **11**, wherein a next iteration of the plurality of iterations of the outer processing loop is initiated responsive to identification of the non-blank CU.

17. The system of claim **11**, wherein the media item comprises a speech utterance, and wherein the representation of the media item comprises a transcription of the speech utterance.

18. The system of claim **11**, wherein the plurality of iterations of the outer processing loop to identify the plurality of CUs of the media item are performed in parallel to a second plurality of iterations of the outer processing loop performed to identify a second plurality of CUs of a second media item, and wherein the plurality of iterations and the second plurality of iterations comprise an equal number of calls to the first NN to identify an equal number of CUs.

19. The system of claim **11**, wherein the system is comprised in at least one of:
an in-vehicle infotainment system for an autonomous or semi-autonomous machine;
a system for performing one or more simulation operations;
a system for performing one or more digital twin operations;
a system for performing light transport simulation;
a system for performing collaborative content creation for 3D assets;
a system for performing one or more deep learning operations;
a system implemented using an edge device;
a system for generating or presenting at least one of virtual reality content, mixed reality content, or augmented reality content;
a system implemented using a robot;
a system for performing one or more conversational AI operations;
a system implementing one or more large language models (LLMs);
a system implementing one or more language models;
a system implementing one or more vision language models (VLMs);
a system implementing one or more multi-modal language models;
a system for performing one or more generative AI operations;
a system for generating synthetic data;
a system incorporating one or more virtual machines (VMs);
a system implemented at least partially in a data center; or
a system implemented at least partially using cloud computing resources.

20. A processing device comprising a processing circuitry to:
identify in parallel, using N calls for batch execution of a predictor network of a transducer speech-to-text model, at least (i) N non-blank units of a first speech utterance and (ii) N non-blank units of a second speech utterance.

* * * * *